
Shared Metadata Service

Persistent Catalog Cache, Language-Service Seam, and STS2 Integration

Design, implementation, and validation review

For [vscode-mssql PR #22836](#)

Central question. How should one shared, key-correct metadata substrate turn SQL catalog rows into immutable, freshness-aware snapshots that can be reused by future language, AI, Object Explorer, Query Studio, and designer features—without putting network I/O on an interactive hot path, cross-publishing one database under another key, losing truth under multi-window cache races, or confusing a generic STS2 query lane with a dedicated metadata endpoint?

Review outcome at the merge-ready head. The architecture and implementation now agree on the load-bearing boundaries. The six prior merge blockers—offline network enforcement, fail-closed database identity plus session recycling, auxiliary-session serialization, cache-key binding, cross-window cache publication ordering, and single serverless-wake ownership—are repaired and covered by deterministic regressions. The branch includes current `main`; activation-snapshot context keys prevent reload-required preview settings from exposing unregistered commands; and bound SQL Data Plane views preserve capability alternatives. Local build, lint, focused tests, and direct packaging are green subject only to two independently reproduced, untouched Windows path-fixture suites.

Prepared for:	Karl Burtram
Primary change:	<code>dev/karlb/query_04_metadata_service</code> at 69ca292f8
Current main merged:	via 37bf23fce from 59c8ea50c; extraction base 69f54be2a
Related vscode ref:	<code>dev/query</code> at 5be148b1e
STS2 reference:	<code>sqltoolsservice dev/query</code> at fe8300939
Refactor notes:	report source parent 176c1494
Report snapshot:	29 August 2026, America/Los_Angeles

This document combines an as-built design explanation with an evidence-qualified implementation review. A companion Markdown artifact contains exact file/line comments, suggested patches, and regression tests suitable for direct PR review or Codex execution.

Contents

How to read this report	1
1 Review snapshot and evidence model	2
2 Executive design summary	3
3 Scope and position in the refactor train	5
4 Runtime architecture and composition	6
5 Identity, authentication, and lease lifecycle	7
6 Catalog model: compact truth with explicit unknowns	9
7 Freshness, drift, and polling	11
8 Persistent cache design	13
9 Architectural Diagram Gallery	18
10 Language-service fit	26
11 STS2 integration and endpoint evolution	27
12 Logging, diagnostics, and privacy	29
13 Test design and coverage	31
14 Build and validation record	34
15 Performance mechanisms and evidence quality	37
16 Mechanism-to-evidence traceability	38
17 Known boundaries and next validation gates	38
18 Final assessment	40
Evidence and source map	40

How to read this report

This report distinguishes five kinds of statement so that design intent, current code, and validation evidence are not accidentally merged:

Implemented

Present in the pinned PR head and traced to its implementation.

Target invariant

A safety or behavior contract the substrate is intended to enforce.

Validated Exercised by a named test, protocol scenario, build lane, or independently observed repository state.

Reviewed gap

A current-head mismatch between the target invariant and the implementation.

Future contract

A downstream consumer or STS2 endpoint shape described by the refactor plans, but not shipped by PR #22836.

Key takeaway

PR #22836 supplies a reusable *metadata substrate*: catalog representation, live hydration, store/lease lifecycle, freshness policies, drift detection, optional persistence, preview composition, and observability. It deliberately does **not** replace the current language service, Object Explorer, or query experience. Today it reaches SQL Server through the existing STS2 connection/query protocol. A future dedicated STS2 metadata service is an adapter substitution, not a fact already delivered by this PR.

Evidence boundary

The source review is pinned to exact commits and current GitHub state. This report combines two evidence sets: independent local builds/tests already run at the pinned head, and a later static cross-repository review of the implementation and protocol contracts. GitHub Actions facts, local results, PR-author claims, review-machine measurements, and proposed-but-not-yet-run regressions are labeled separately rather than presented as one undifferentiated proof.

1 Review snapshot and evidence model

Table 1: Pinned sources used to reconstruct the feature and its surrounding refactor.

Source	Pinned state	Use in this report
PR #22836	merge-ready head 69ca292f8; current-main merge 37bf23fce with par- ent 59c8ea50c; extraction base 69f54be2a	Current changed filenames, review discussions, Actions state, and exact line anchors.
vscode-mssql dev/query	5be148b1e	Related query/data-plane refactor history and already- merged platform seams.
vscode-mssql dev/refactor	remote ref at a0880ad4f	Integration-tree comparison and downstream refactor context; the branch was published during this closeout so the reference is reproducible.
sqltoolsservice dev/query	fe8300939	STS2 generated contract, invariants, runtime behavior, and executable one-active-query scenario.
kburtram/notes main	source parent f56db6d73	Metadata substrate/cache/drift, language-service, STS2, and observability design intent.

Evidence is ranked as follows:

- Pinned source and executable protocol contracts.** These establish control flow, ownership, serialization, identity, and persistence behavior.
- Independent local validation.** The extension, embedded perftest, sqltoolsservice solution, STS2, authentication, language, and ServiceLayer lanes were run locally; exact results and exclusions appear in [table 11](#).
- GitHub Actions logs.** At historical head 9955ff9e6, analysis, packaging, normalization, and platform launch checks were green; the aggregate build-and-test check was red even though its MSSQL lane reported 3,975 passing, 0 failing, and 12 skipped. The aggregate failure came from SQL Projects unit exit code 1 and one of 17 VSIX smoke tests. At merge parent 37bf23fce, build/test and five of six platform-launch jobs passed; win-x64 failed before artifact download or product launch when `npm ci` encountered Windows `EPERM` cleanup errors under `monaco-editor` and then failed git-dependency preparation. At the final 69ca292f8 closeout snapshot, analysis, aggregate CodeQL, normalization, CLA, MSSQL VSIX packaging, aggregate build/test, and five of six platform launches were green. Win-arm64 launch and the separate Azure PR validation context remained in progress. This artifact records observed results only and does not predict pending conclusions.
- PR-body and prior-review results.** Additional focused pass counts, packaging measurements, and review-machine timing probes are useful reported evidence and remain identified as such.
- Closure regressions.** The repaired head adds the adversarial branches that were missing: offline hit/miss/refresh/strict/auxiliary/transition paths, identity mismatch, auxiliary overlap, key-copy and logical-hash corruption, older-last and manifest-only publication races, abort-listener balance, unknown case semantics, provider availability, and server hydration timeout/recycle.

Current review disposition

The design remains the basis of the refactor and the repaired head is suitable for merge review. The original blockers and hardening findings in [section 17.4](#) are closed in code and tests. The final six-minute delayed recheck reported 52 GitHub review threads, zero unresolved, exact head 69ca292f8,

and a mergeable branch. The remaining cross-window index caveat is explicit by design: access timestamps and LRU order are approximate across windows, while manifests remain authoritative and maintenance rebuilds membership and byte totals.

2 Executive design summary

The feature addresses a real architectural problem. Without a common substrate, each metadata-backed surface tends to grow its own connection, query set, cache, name-resolution behavior, freshness policy, and failure semantics. That duplication is expensive, but the larger risk is semantic disagreement: completion may believe a column exists while Object Explorer, AI context, or scripting sees a different generation or silently treats unavailable data as an empty catalog.

PR #22836 establishes one extension-host registry with two scopes:

- a **server catalog** keyed by a non-reversible server fingerprint; and
- a **database catalog** keyed by that fingerprint plus the requested database spelling.

Consumers acquire ref-counted leases. A database lease exposes a current immutable snapshot, deterministic name/search/schema-context APIs, lazy module definitions, DDL notification, refresh, auxiliary sections, and policy-routed freshness. The mutable builder, filesystem, credential store, and VS Code configuration remain behind composition boundaries.

The live engine uses the SQL Data Plane to run a progressive H0–H7 catalog ladder. Rows are accumulated in a compact structure-of-arrays builder and published atomically as a **CatalogSnapshot** with a process-local generation and explicit per-section readiness. The persistent cache serializes the same model into canonical, gzip-compressed, content-addressed payloads with a manifest, bounds, and privacy policy intersection.

Table 2: Executive scorecard: design target versus the pinned implementation.

Area	Target contract	Current-head assessment
Model and publication	Immutable, deterministic generations with per-section truth.	Implemented Strong. Structure-of-arrays, deterministic projections, partial readiness, and scale fixes are well covered.
Shared lifecycle	One entry per key, ref-counted leases, bounded idle retention, complete disposal.	Implemented Strong after same-key single-flight and disposal fixes.
Database isolation	A requested key can publish only meta-data proven to belong to that database; recovery must reopen a fresh session.	Implemented H0 mismatch fails before H1/publication, comparison follows known catalog semantics, and lazy sources forward <code>recycle()</code> and reject late opens after disposal.
Freshness/offline	Consumers choose stale, validated, live, or network-forbidden behavior.	Validated A live gate now spans resolver, hydration, validation, poll/retry timers, strict refresh, lazy definitions, server scope, and auxiliary work; hit, miss, and online→offline regressions assert zero new resolver/session/query calls.
STS2 query ownership	One active query per physical session, including lazy auxiliary sections.	Validated All section keys share one FIFO lane, and both database and server hydration watchdogs fence abandoned results and recycle before retry.
Cache integrity	A payload is bound to its key, logical hash, and a globally ordered publication decision.	Validated Reads verify requested-key binding and decoded canonical content hash; per-key cross-process publication locks serialize compare/write/postcheck using capture time plus deterministic equal-time tie-break.
Data-plane retry policy	Serverless wake/retry has one owner per open attempt.	Validated The public service view is the sole wake owner; top-level open delegates once, and provider availability refusal is a retryable <code>unavailable</code> , not a capability failure.
Consumer cutover	Language/AI/OE/Query Studio pin snapshots and degrade honestly.	Future contract No ordinary product surface is routed to the substrate in this PR.

Design invariant

The load-bearing target is **honest degradation**: missing, failed, partial, stale, offline, identity-drifted, or cache-invalid metadata must remain distinguishable all the way to the consumer. An empty list, an old generation, or a successful event must never be used as a convenient substitute for “not known.”

Why the repairs were required

The six P1 findings could have caused live network activity under an offline promise, wrong-database publication, nondeterministic auxiliary failures, copied cache entries crossing keys, older-window authority, or nested wake loops. The repaired head closes those substrate-boundary failures rather than asking downstream consumers to compensate. The regression matrix in [table 10](#) is the durable proof, not merely the absence of a failure in a happy-path suite.

3 Scope and position in the refactor train

3.1 What lands in this PR

The 46-file change spans the model, live engine, shared store, persistent cache, host/data-plane wiring, localization, observability, focused tests, and the embedded perftest registry. Its center of gravity is five mechanisms:

1. **Catalog model.** Compact immutable snapshots, collation-aware resolution, deterministic projections, exact column/FK detail, readiness, and statistics.
2. **Live metadata engine.** Progressive hydration, dedicated sessions, lazy definitions, validation, DDL sniffing, digest polling, identity drift, focus/serverless governance, and retry containment.
3. **Shared store.** Server/database keys, single-flight first acquisition, ref-counted leases, bounded idle retention, auxiliary catalogs, and one extension-host composition root.
4. **Persistent cache.** Canonical codec, manifest, filesystem protocol, coordinator, policy intersection, corruption handling, multi-window behavior, and age/byte eviction.
5. **Host integration.** Private-preview gates, settings/commands/localization, SQL Data Plane hardening, shutdown flush, observability registration, and a perftest warm-acquire probe.

3.2 What does not land

The PR does not route completion, hover, diagnostics, scripting, Query Studio, Object Explorer v2, or AI context through the new store. It also does not add a native TypeScript language engine, dedicated metadata RPCs to STS2, replay bundles, or central diagnostic upload. Those are downstream consumers or platform increments.

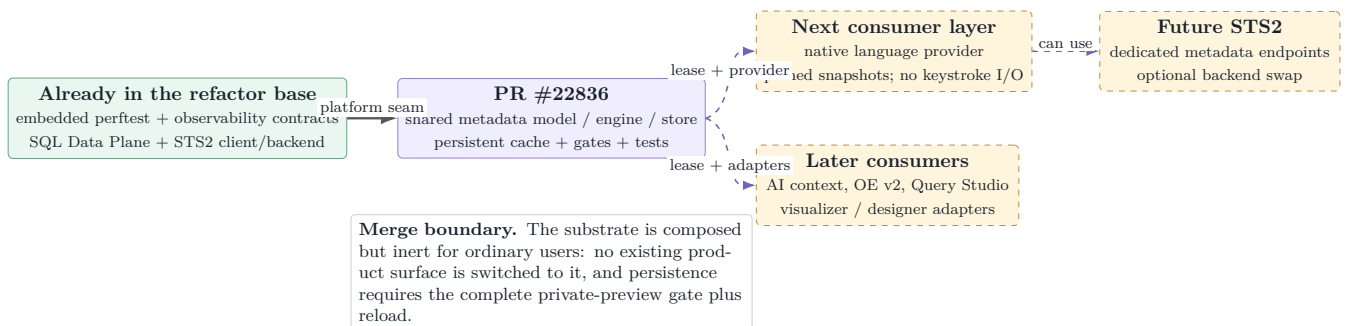


Figure 1: The PR is an enabling layer in the refactor train, not the consumer cutover. Dashed arrows are downstream contracts rather than current product routing.

4 Runtime architecture and composition

4.1 One extension-host composition root

`MainController.initializeMetadataStore()` configures host facts and storage before the first store request. `MetadataStoreService` lazily creates one `MetadataStore` per extension host. The metadata engine itself imports neither VS Code nor the filesystem; focus, poll cadence, data-plane enablement, global-storage path, and settings are injected.

The hierarchy of gates is strict:

```
cache composed  $\iff$  enableExperimentalFeatures
                     $\wedge$  sqlDataPlane.enabled
                     $\wedge$  metadataCache.enabled.
```

All three settings default off where applicable, and changing the dependency path requires a window reload. SQL Data Plane launch arguments also use the private-preview predicate, so merely setting the lower-level flag without the experimental umbrella does not activate STS2.

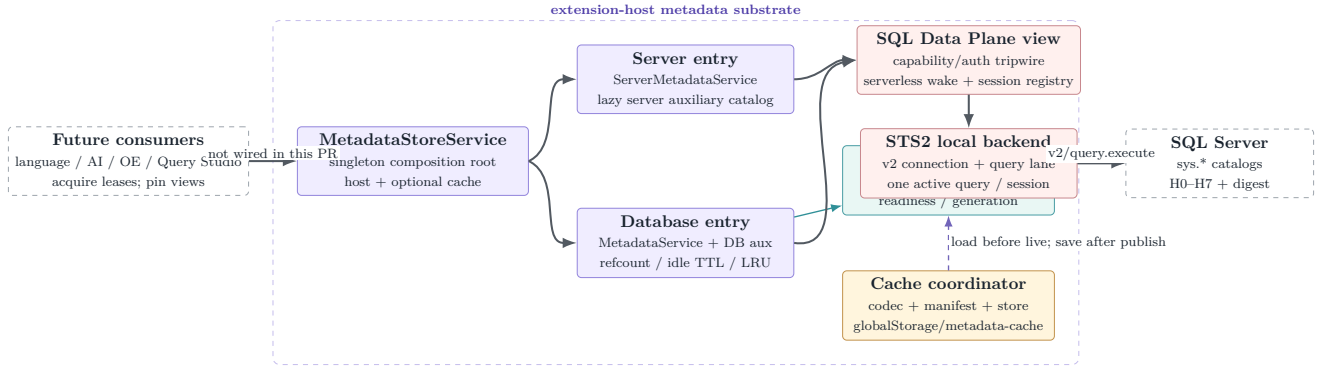


Figure 2: As-built runtime layering. The current transport is generic STS2 query execution; the cache is optional; consumer arrows remain a future integration surface.

4.2 Acquisition and the network-authorization boundary

The acquisition path has two logically separate responsibilities: recover the best local snapshot and, only when policy permits, create live infrastructure. The target ordering is cache-first and network-gated. This matters even when the backend does not open a physical SQL connection immediately: backend startup, polling, strict freshness, and auxiliary calls are all forms of live work that an explicit offline mode must prohibit.

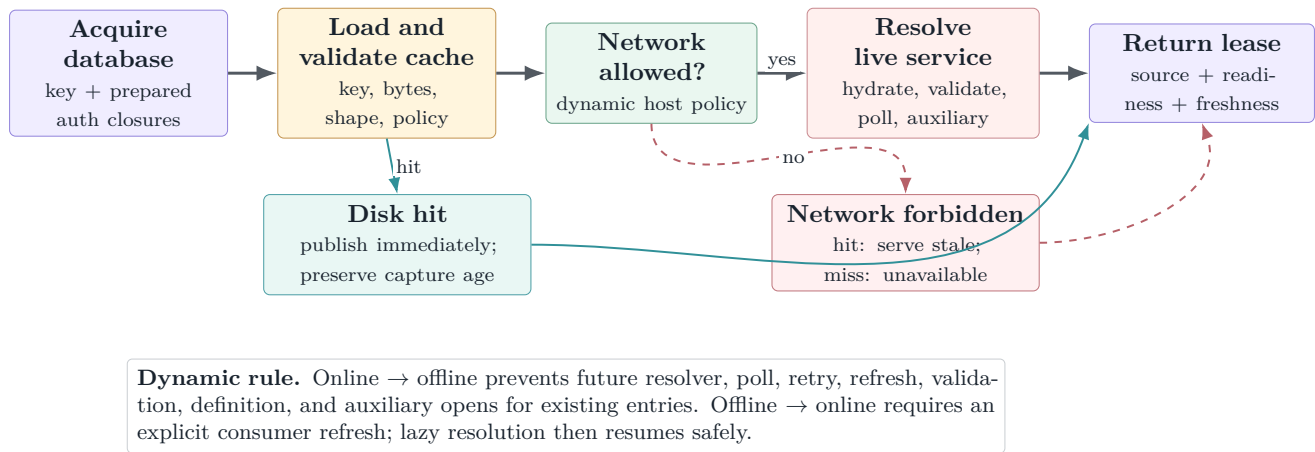


Figure 3: Implemented acquisition ordering. Offline is a network authorization boundary, not merely “do not schedule the first background refresh.”

Current-head review finding

The repaired store uses a shared lazy provider resolution and passes a live authorization closure into database engines, server engines, and both auxiliary sources. Cache hit, cache miss, explicit refresh, strict freshness, definitions, polling/retry, and online→offline paths are exercised with resolver/session/query counters. Disposal is rechecked after an asynchronous provider resolution so a late result cannot open a session after ownership ended.

4.3 Activation, maintenance, and shutdown

Composition is deliberately cheap. Cache maintenance is scheduled five seconds after activation and therefore does not contaminate `mssql.activate` timing. Debounced cache writes are flushed during extension deactivation before store disposal. Status commands are passive views of already-composed state; the SQL Data Plane status command is designed not to construct a backend or resolve credentials.

The cache-specific commands are:

- `mssql.metadataCache.showStatus;`
- `mssql.metadataCache.clearAll;`
- `mssql.metadataCache.clearForConnection;` and
- explicit enable/disable commands for offline mode.

5 Identity, authentication, and lease lifecycle

5.1 Two non-reversible fingerprints

The store never uses a password, token, or connection string as identity. It JSON-encodes connection-affecting tuples and hashes them with SHA-256, returning a 22-character base64url prefix:

- `pfp_*`: profile/session identity, including the default database;
- `sfp_*`: server identity, excluding the database.

JSON tuple encoding matters: delimiter-joined fields allow distinct legal values such as `["a|b","c"]` and `["a","b|c"]` to collide. The current PR fixes and tests this class for both fingerprints and derived profile IDs.

The database store key is `serverFingerprint + requested database spelling`. Retaining the spelling is reasonable before the catalog comparison rule is known, but it creates an obligation: the live session must prove that the database it opened is the database represented by the key before any snapshot or cache write is published.

5.2 Secrets stay deferred

`prepareConnection()` produces a sanitized profile reference and closures for authentication:

- SQL passwords are read from the credential store only when a physical session open requires them;
- Entra tokens are acquired only at physical open;
- integrated authentication carries no secret provider.

The SQL Data Plane registry evaluates declared requirements before invoking credential closures. The repaired public service view is the single Azure serverless wake owner; top-level `openSession()` delegates through that view once. Static requirement failures remain non-retryable capability errors, while a live provider `canOpen()` refusal becomes retryable `unavailable`. This keeps capability checking, credential deferral, provider availability, and wake ownership separate.

5.3 Single-flight entries and ref-counted leases

First acquisition is single-flight per key for both server and database entries. Concurrent same-key callers join one creation rather than producing orphan engines or sessions. Each lease increments a refcount; disposal decrements it idempotently. A zero-ref database remains warm for a bounded idle period, and the least recently released idle entry is disposed first when the cap is exceeded.

A database entry owns:

- one session source for serialized hydration, digest validation, and lazy module definitions;
- one separate source for database auxiliary sections;
- one metadata engine, one lease handle, listeners, and cache-source state.

The resource split keeps ordinary metadata work away from user query sessions. A catalog-wide FIFO lane now serializes different auxiliary section keys around session open and query completion, while same-section callers still coalesce. The deterministic fake-backend regression starts two distinct sections together and proves both complete through one STS2-compatible one-active-query session.

5.4 Fail-closed database identity and recovery

The target identity protocol has two stages. The returned session metadata is a preliminary signal; H0's `DB_NAME()` plus `CATALOG_DEFAULT` comparison behavior is the authoritative proof. No H1–H7 rows should publish until that proof succeeds. A timeout or later rename drift must fence the current operation epoch, recycle the physical source, and reopen by the requested database name before clearing the drift latch.

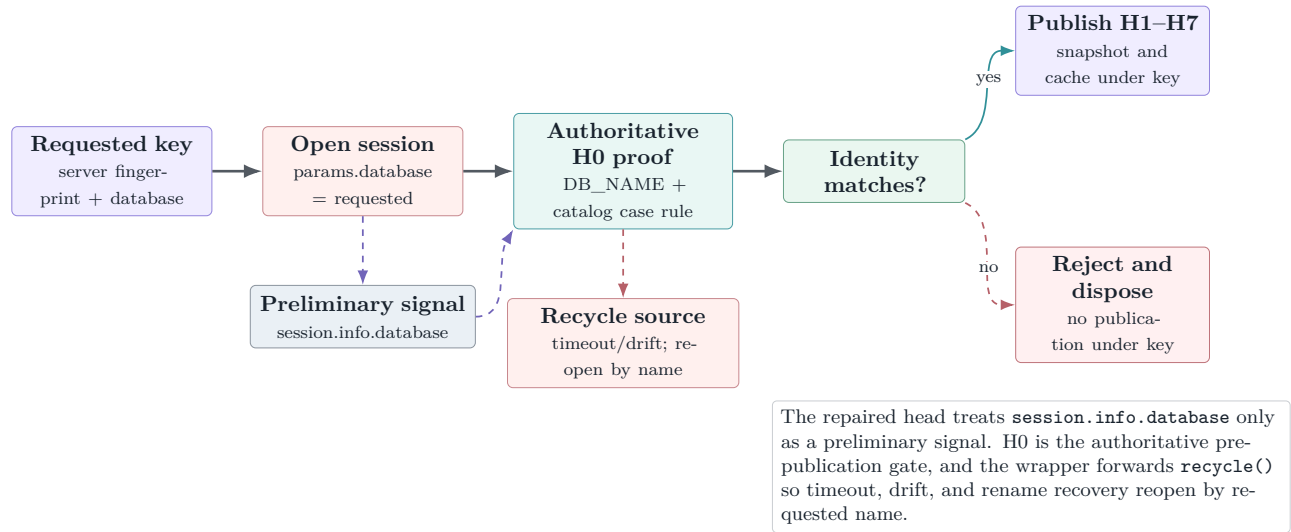


Figure 4: Database identity must be a publication gate. Detection without rejection is observability, not key correctness.

Current-head review finding

This gate is implemented, not aspirational: the wrapper forwards `recycle: () => inner.recycle()`, and authoritative H0 identity must match the requested key under the catalog comparison rule before H1 or cache publication. Tests cover a backend that ignores `params.database`, case-insensitive spelling variation, a true mismatch, and watchdog recovery proving that the next operation uses a fresh session.

6 Catalog model: compact truth with explicit unknowns

6.1 Structure of arrays and immutable publication

`CatalogBuilder` interns strings and appends parallel arrays for schemas, objects, columns, keys, foreign keys, parameters, descriptions, and exact details. An object-id index makes row ingestion constant-time per row. `CatalogSnapshot` builds read-only indexes over those arrays and exposes query methods, not mutation.

This design has three consequences:

1. repeated schema/object/type names occupy one string-table entry;
2. canonical array order is directly serializable and hashable; and
3. a published generation is internally consistent because readers never observe a partially mutated model.

The exact detail surface includes stable `column_id`, type schema/base type, system and user type IDs, UDT/assembly flags, raw max length in bytes, precision/scale, collation, default definition, exact-text identity seed/increment, computed definition/persistence, FK constraint IDs, referential actions, column-pair ordinals, and endpoint column IDs. Defaults and computed definitions are live structural detail but are stripped before persistence.

6.2 Readiness is per section

Each section is one of **absent**, **loading**, **ready**, **failed**, **stale**, or **lite**. The snapshot also carries **full**, **partial**, or **lite** mode. The distinction is semantic:

- **absent**: not hydrated or intentionally excluded;
- **failed**: attempted but unavailable;
- **lite**: useful identity-level data but not the complete contract;
- **stale** in readiness: a transient rehydration state over an existing snapshot, *not* age-based staleness.

Age and validation are represented by the freshness result described in [section 7](#). This prevents one overloaded boolean from lying about both completeness and recency.

6.3 Progressive hydration ladder

Table 3: The live database hydration ladder at the pinned PR head.

Step	Section/fact	Important behavior
H0	Environment	Engine edition, default schema, database collation, <code>CATALOG_DEFAULT</code> case rule, canonical <code>DB_NAME()</code> . Current fallback silently uses case-insensitive lookup when the probe fails; the safer contract is unknown/exact-only or an unavailable name-resolution section.
H1	Schemas	All visible <code>sys.schemas</code> ; no high-ID role-schema filter.
H2	Objects	User tables, views, procedures, scalar/table functions, synonyms, CLR routines/aggregates; system-shipped rows excluded.
H3	Columns/details	Visible columns only; hidden temporal/graph implementation columns excluded; exact type/default/identity/computed facts retained live.
H4	PK/UQ	Primary-key and unique-constraint columns in key ordinal order.
H5/H5B	Foreign keys	Enabled FKs, including untrusted-but-enforced constraints; referential actions and ordered column pairs; invisible endpoints are not published as dangling edges.
H6	Parameters	Routine parameters and return value ordinal 0; schema-qualified UDT display retains exact spelling.
H7	Descriptions	Object/column <code>MS_Description</code> values, failure-visible and live-only under current persistence policy.

Synonyms are honestly **lite**: identity is present, target traversal is not. The `types` section is **absent**: table-type shapes and sequences remain a downstream model enrichment. Module text is not part of H0–H7; `getModuleDefinition(objectId)` performs one safe-integer-checked, lazy `sys.sql_modules` query and caches it only for the current generation.

6.4 Server and auxiliary catalogs

The server service hydrates environment and database-list facts independently from database catalogs. Additional server and database folders use **AuxiliaryCatalog**: a keyed table of lazy sections with single-flight hydration and per-section refresh. This is the extension point for security, broker, storage, programmability leaves, logins, roles, credentials, endpoints, system objects, and designer facets without bloating connect-time hydration.

Auxiliary queries use sources separate from the main hydration lane, which is the correct resource boundary. The repaired auxiliary engine retains same-section coalescing and adds a catalog-wide FIFO promise lane around `sessions.open()` plus `runMetadataQuery()`, so different section keys cannot overlap on one cached STS2 session. Names returned by auxiliary sections remain outside diagnostic events.

6.5 Deterministic schema context

`buildSchemaContext()` produces bounded text from a pinned snapshot for AI or language consumers. Ordering uses an ordinal, locale-independent comparator: Unicode-default fold followed by a raw code-unit tiebreak. This avoids Electron/ICU-version drift and preserves case-distinct names. The selection algorithm precomputes FK degree, renders each candidate at most once per fidelity tier, and takes a prefix that fits the character budget; the former quadratic shrink-and-rerender path is now linear in catalog size per tier.

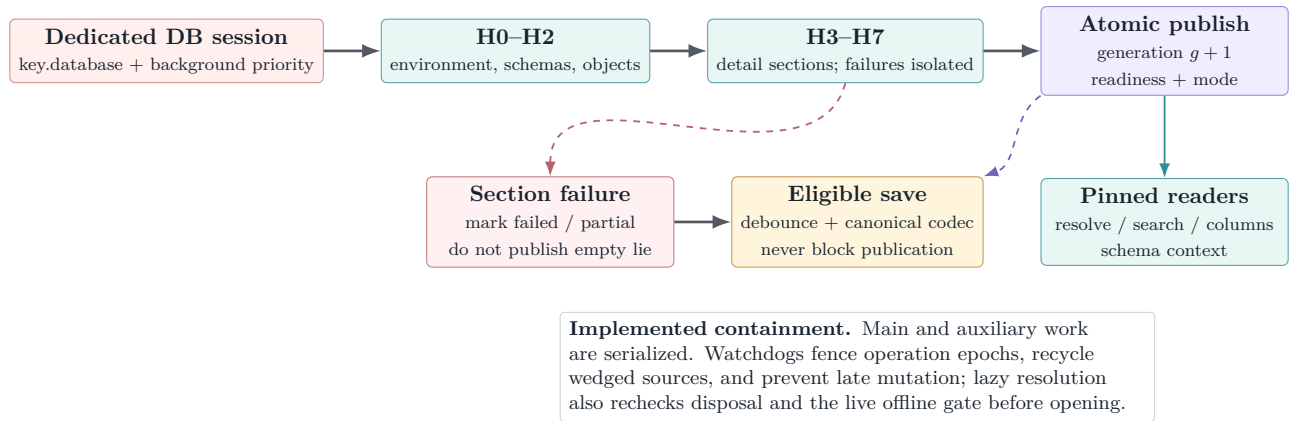


Figure 5: Rows are accumulated privately and published as one immutable generation. Per-section failure truth survives publication and persistence eligibility.

7 Freshness, drift, and polling

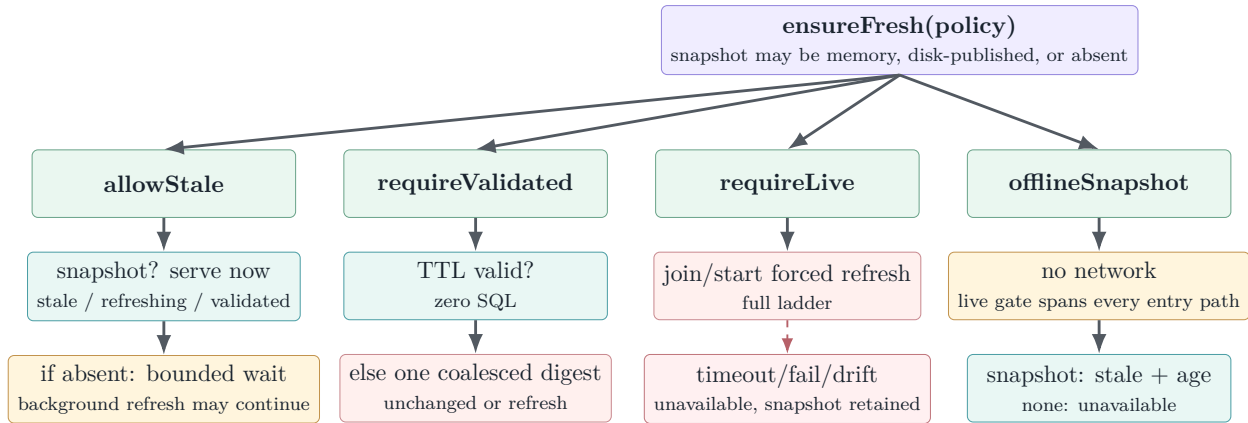
7.1 Consumers state a policy, not an implementation

The freshness API decouples consumer intent from cache and network mechanics. A request declares a mode, reason, optional required sections/objects, staleness or validation TTL, partial-data tolerance, disk-load permission, background-refresh preference, and a wait/abort budget.

Table 4: Freshness modes and their honest outcomes.

Mode	Normal use	Decision semantics
<code>allowStale</code>	completion, AI context	Return any snapshot immediately; optionally start background refresh. With no snapshot, join hydration only up to the caller's budget.
<code>requireValidated</code>	diagnostics, OE browse	Accept a recent validation TTL; otherwise coalesce a cheap digest. Unchanged means validated; changed chains a full refresh. Timeout returns best snapshot as stale/unavailable, never cancels shared work.
<code>requireLive</code>	scripting or destructive/fidelity-sensitive work	Join/start a forced refresh. Failure, timeout, or identity drift returns unavailable ; a retained snapshot may still ride in the result for an explicit offline choice.
<code>offlineSnapshot</code>	user-selected offline operation	Perform no backend resolution, SQL session open, poll, validation, refresh, or auxiliary query. Return disk/memory snapshot as stale with age, or unavailable if none exists. The live store/engine/server/auxiliary gate enforces the same rule after an online→offline transition.

Freshness results distinguish **live**, **validated**, **refreshing**, **stale**, and **unavailable**; they also carry source, generation, capture time, stale age, wait time, validation summary, and whether background work started. Section readiness is checked *after* the freshness decision. A strict caller requiring columns cannot accept a fresh snapshot whose columns section failed.



Shared-work rule. Timeouts and aborts stop only this caller's wait. They never cancel a coalesced hydration or validation needed by other leases. Readiness gates then decide whether requested sections satisfy the caller.

Figure 6: The freshness decision has four modes but one invariant: return the best snapshot with an explicit truth label; never silently elevate stale or partial data.

7.2 Three drift paths

Local DDL sniff. After a successful executed batch, a leading-keyword lexer recognizes **CREATE**, **ALTER**, **DROP**, and **SP_RENAME**; **EXEC/EXECUTE** trigger a conservative digest check. A burst while hydration is active collapses to at most one follow-up refresh.

Cheap digest. The T1 query hashes visible object count plus object ID, schema ID, byte-exact name, and modify date. Including schema and binary name fixes the classic `sp_rename/ALTER SCHEMA TRANSFER`

blind spot. It remains evidence of likely equality, not proof: column/key/FK/parameter-only drift awaits future section/object digests or an explicit refresh.

Explicit policy. A strict consumer can force validation or a full live refresh. This is the correctness backstop when polling is disabled, suspended, or intentionally sparse.

7.3 Poll governance

The default base interval is 60 seconds with multipliers 1, 2, 5 and independent $\pm 10\%$ jitter. Consecutive unchanged verdicts back off; refresh or detected activity resets to base. Validation fan-out is capped at two per server fingerprint, so one sleeping server cannot starve another and a resume event cannot open dozens of digest queries against one server.

After two minutes unfocused, polling suspends without SQL. Refocus schedules immediate, limiter-bounded validation and re-jitters timers. Azure SQL Database and Synapse serverless engine editions (5 and 11) poll at the cap so a background digest is less likely to defeat auto-pause; consumer activity still invokes TTL validation. Setting `mssql.metadata.pollSeconds=0` disables polling.

8 Persistent cache design

8.1 Purpose, defaults, and trust boundary

The cache reduces first-use latency and supports explicit offline browsing; it is not the source of authority. It is off by default, stored under the extension's local or remote `globalStorage/metadata-cache/v1`, limited by default to 14 days, 256 MiB total, and 32 MiB compressed per entry, with a five-second save debounce.

A cache entry is attacker-shaped or corruption-shaped local input. The read path must therefore prove four independent properties before publication:

1. **key ownership:** the manifest belongs to the requested server/database key;
2. **physical integrity:** referenced compressed bytes exist, fit bounds, and match SHA-256;
3. **logical integrity:** decoded canonical content matches the manifest's content hash and structural invariants;
4. **policy compatibility:** privacy/readiness is intersected with current settings rather than elevated.

The repaired implementation verifies all four properties. Requested server fingerprint, derived database hash, and optional exact database spelling must match before adoption; decoded payloads are canonicalized and re-hashed after shape validation. Mismatch is a classified safe miss with best-effort quarantine.

8.2 Canonical payload and manifest

The `cm2` payload is canonical-order JSON over environment, interned strings, and structure-of-arrays fields, gzip-compressed. A model-version change is a clean miss rather than an in-place migration. Two hashes serve different jobs:

- `manifest.payload.sha256`: full SHA-256 of the compressed payload file;
- `contentHash`: `csh_` plus a truncated base64url SHA-256 of canonical uncompressed JSON, intended to identify logical equality across processes/compressions.

The manifest records producer versions, writer ID, server fingerprint, database hash and optional exact database name, capture/generation/source, validation summary, environment, readiness, mode, counts/sizes, privacy policy, and the content-addressed payload filename.

Manifest key binding

`readEntry(key)` now compares `manifest.key.serverFingerprint`, `databaseHash`, and optional `databaseExact` with the requested key. The regression copies a fully valid A entry into B's already-derived directory and proves B returns `keyMismatch` and invalidates the manifest rather than adopting A's catalog.

8.3 Privacy intersection

Current declared settings force description and module-definition persistence off. The codec strips default/computed SQL definitions before writing, and row counts are absent. The disk payload therefore contains structural catalog facts: schema/object/column/type/key/FK/parameter identities and exact non-definition facets.

Load applies policy intersection in both directions. A section disallowed by current policy, absent from payload, or never supported in v1 becomes **absent**; it never becomes ready-but-empty. A policy-ID mismatch is a normal intersected hit, not corruption.

Validity warning

“Hashed paths” are not encryption. The manifest may contain the exact database name, and the payload contains structural names and types. This is sanctioned local metadata, potentially sensitive, under VS Code global storage. The contract is exclusion, integrity, boundedness, and safe diagnostics—not confidentiality from a user or process that can read the directory.

8.4 Read protocol

A reader begins with the manifest because it is the trust root. It validates format/codec/model/shape, resolves the content-addressed payload, checks actual compressed size, verifies SHA-256, enforces a hard decompressed bound during gunzip, parses JSON, validates parallel-array and referential invariants, intersects policy, adopts arrays, and preserves the original capture time/generation.

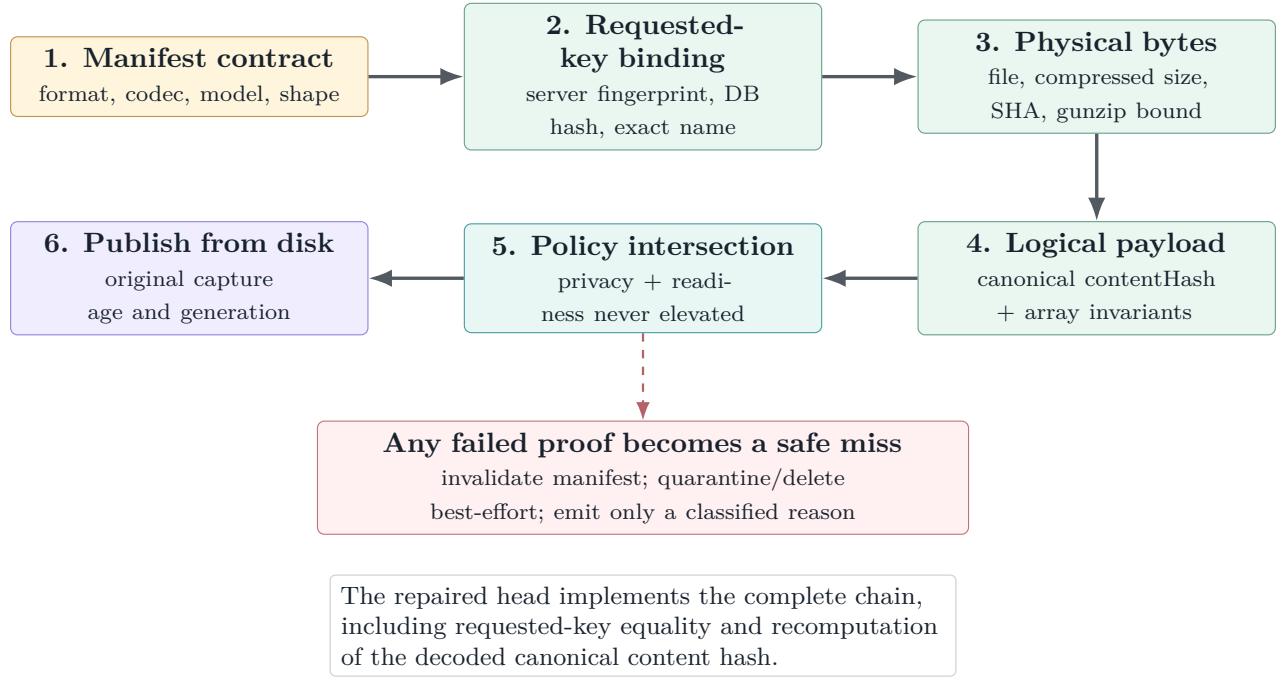


Figure 7: Complete cache trust chain. A structurally valid manifest is not trusted merely because it sits in the expected directory.

8.5 Write protocol and cross-window authority

Payloads are content-addressed. A writer serializes and compresses, checks the cap, writes a same-directory temporary file with fsync, renames the payload into place, then writes and renames the manifest last. Transient Windows rename failures retry with bounded jitter. Content addressing prevents a manifest from accidentally referring to another writer's bytes with the same fixed filename.

Content addressing does not, by itself, decide which capture is authoritative. The repaired coordinator adds an exclusive-create per-key publication lock around re-read, authority comparison, payload/manifest commit, and postcheck. Authority uses the globally comparable capture instant, never a process-local generation; equal-instant unequal payloads use the ordinal content hash as a deterministic tie-break.

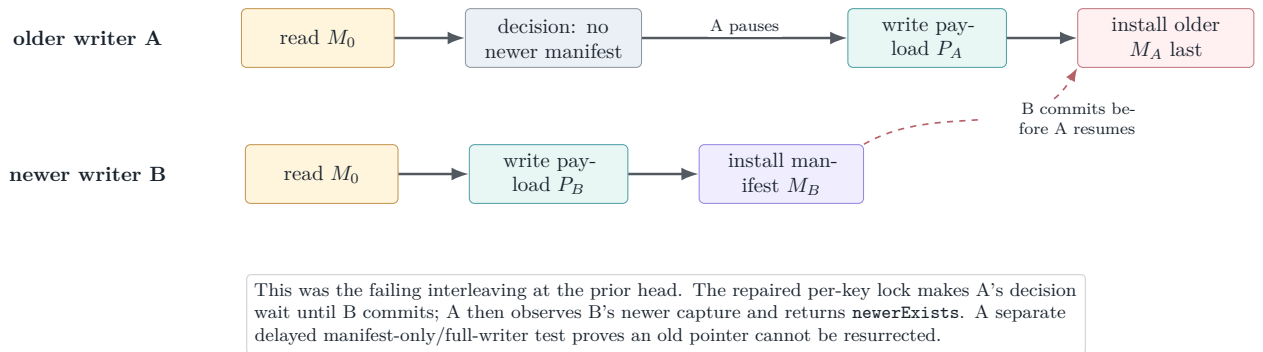


Figure 8: Formerly unsafe two-window interleaving, now retained as a deterministic regression. Content addressing protects bytes; the per-key critical section protects authority.

Table 5: Required cache publication properties.

Property	Repaired mechanism	Closure evidence
Payload/manifest byte coherence	Content-addressed payload + manifest-last + SHA	Strong; retain bounded manifest re-read for reader/writer races.
Newer capture cannot be overwritten	Exclusive per-key lock + capture-time authority + deterministic equal-time hash tie-break	Reverse-order older/newer writer regression; newer capture remains authoritative even with lower local generation.
Manifest-only update cannot resurrect stale pointer	Equality proof and manifest rewrite occur under the same lock using a fully validated payload	Delayed old manifest-only writer interleaves with a newer full writer; new payload remains readable.
Logical equality is trustworthy	Decoded canonical <code>contentHash</code> is recomputed before adoption or manifest-only equality	Valid compressed bytes and shape with a false logical hash are rejected as corrupt.
Reader key isolation	Manifest key fields are compared explicitly to the requested key	Valid A entry copied into B's directory returns <code>keyMismatch</code> .

8.6 Eviction, clearing, and index status

Maintenance coalesces re-entrant calls and throttles filesystem operations. Removal order is corrupt/unsupported and aged entries first, then least-recently-used entries until bytes fit. Manifest-less payloads, old temporary files, quarantine files, and superseded payloads receive an orphan grace. Clears remove manifests first, making remaining bytes inert before deleting payloads.

The JSON index is a convenience cache for listing/access/LRU rather than the trust root. Each window may still lose an access tick while replacing the whole index; source documentation now calls cross-window access/LRU approximate. Manifests remain authoritative for identity and content, and maintenance rebuilds membership and byte totals. Serializing this advisory file with catalog publication would widen a correctness-critical lock for no catalog-safety gain, so this limitation is accepted rather than hidden.

8.7 Composite cache lifecycle and downstream consumer routing

The following view combines the reviewed cache substrate with the downstream consumer wiring present on `upstream/dev/refactor`. It is intentionally broader than the PR boundary: the store, snapshot, persistence, and STS2 paths are the mechanisms reviewed above, while the top-row consumer adapters are later integration work and must not be read as routes shipped by PR #22836 itself.

Composite scope

Consumers never read cache files or execute catalog SQL directly. They acquire leases or pinned views from the shared store. The cache is a verified projection of a published database snapshot; server catalogs and auxiliary sections remain live, lazy memory structures. Solid consumer arrows below describe the downstream `dev/refactor` composition, not the merge boundary of the reviewed PR.

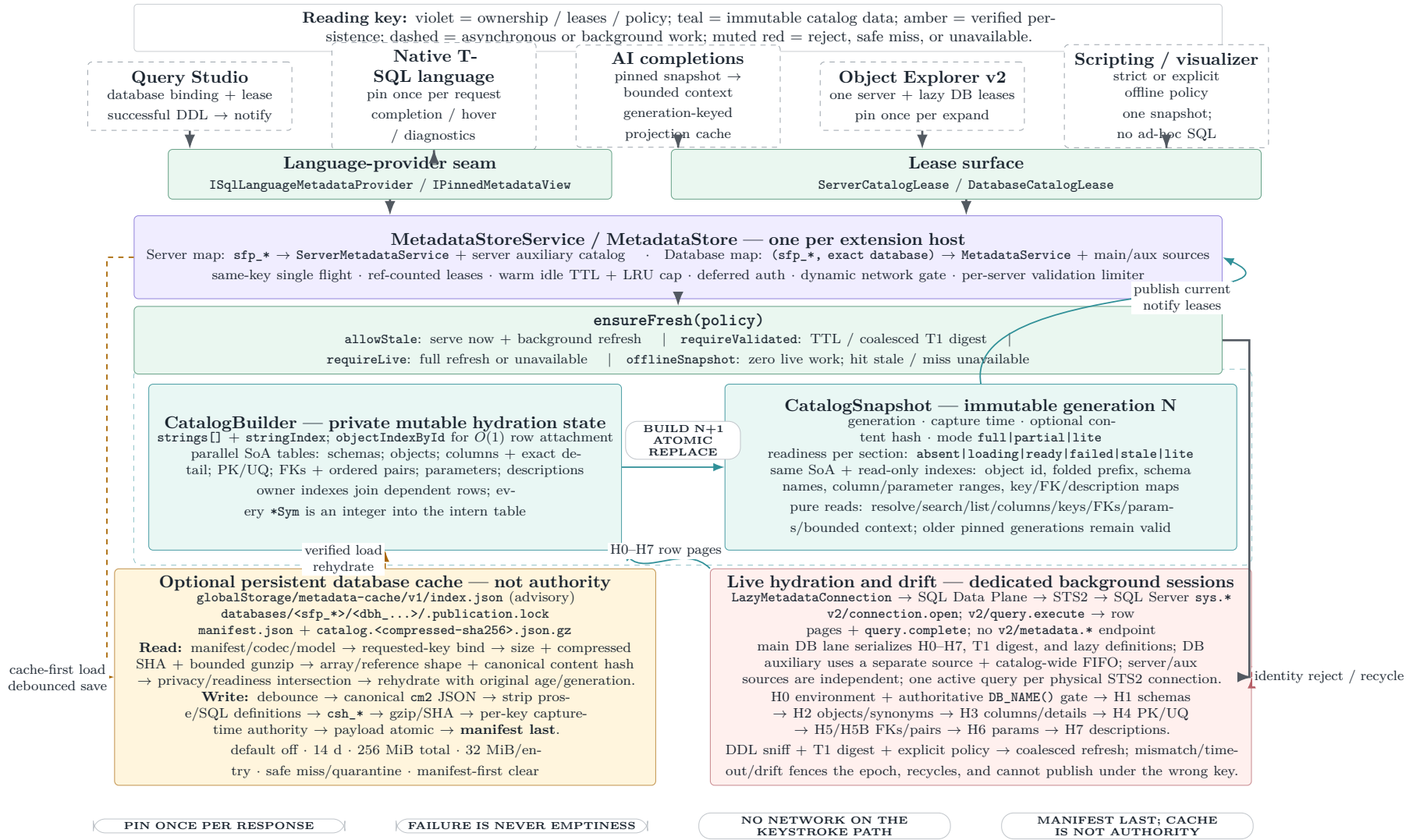


Figure 9: Composite memory, persistence, and consumer flow. The lower layers are the reviewed PR #22836 substrate. The consumer routes at the top are traced from the downstream **dev/refactor** integration and preserve the same invariants: consumers pin one immutable generation, express freshness as policy, and never bypass the shared store.

9 Architectural Diagram Gallery

The composite in [figure 9](#) is the report-native architectural reference. This gallery retains two alternate composite renderings and the complete focused diagram set so that the same design can be reviewed at three levels: one-canvas synthesis, four-part index, and full-page vector detail. All assets, editable sources, the generator, the original archive, and the image-generation prompt live under `figures/metadata_cache/`. The `README.md` in that directory records their provenance and intended use.

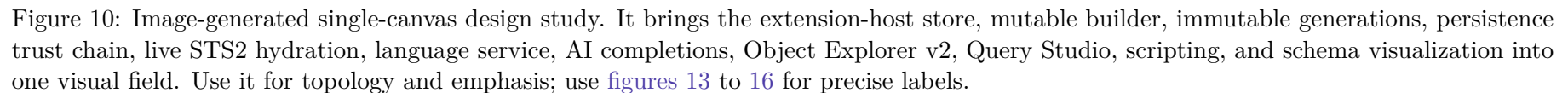
Rendering and evidence boundary

The alternate composites are visual design studies, not independent implementation evidence. Small generated labels can drift even when the overall topology is useful. If a rendered label conflicts with the reviewed source, tests, report prose, or deterministic TikZ figures, those reviewed artifacts control. The four focused diagrams are included from their vector PDF exports; equivalent SVG and 3840-by-2160 PNG files are retained beside them for editing and reuse.

Table 6: Gallery reading map.

Rendering	Best use	What it emphasizes
Single-canvas study, figure 10	Fast cross-layer scan	Consumer seams, ownership, memory generations, hydration, persistence, and invariants in one dense field.
Tall composite, figure 11	Continuous scrolling	A top-to-bottom progression from integration surface through memory and operation to disk protocol.
Contact sheet, figure 12	Side-by-side comparison	The four focused views at a common scale, useful as an index and review checklist.
Focused vector set, figures 13 to 16	Detailed inspection	Full-size, zoomable labels and one concern per page; preferred when validating a specific mechanism.

19



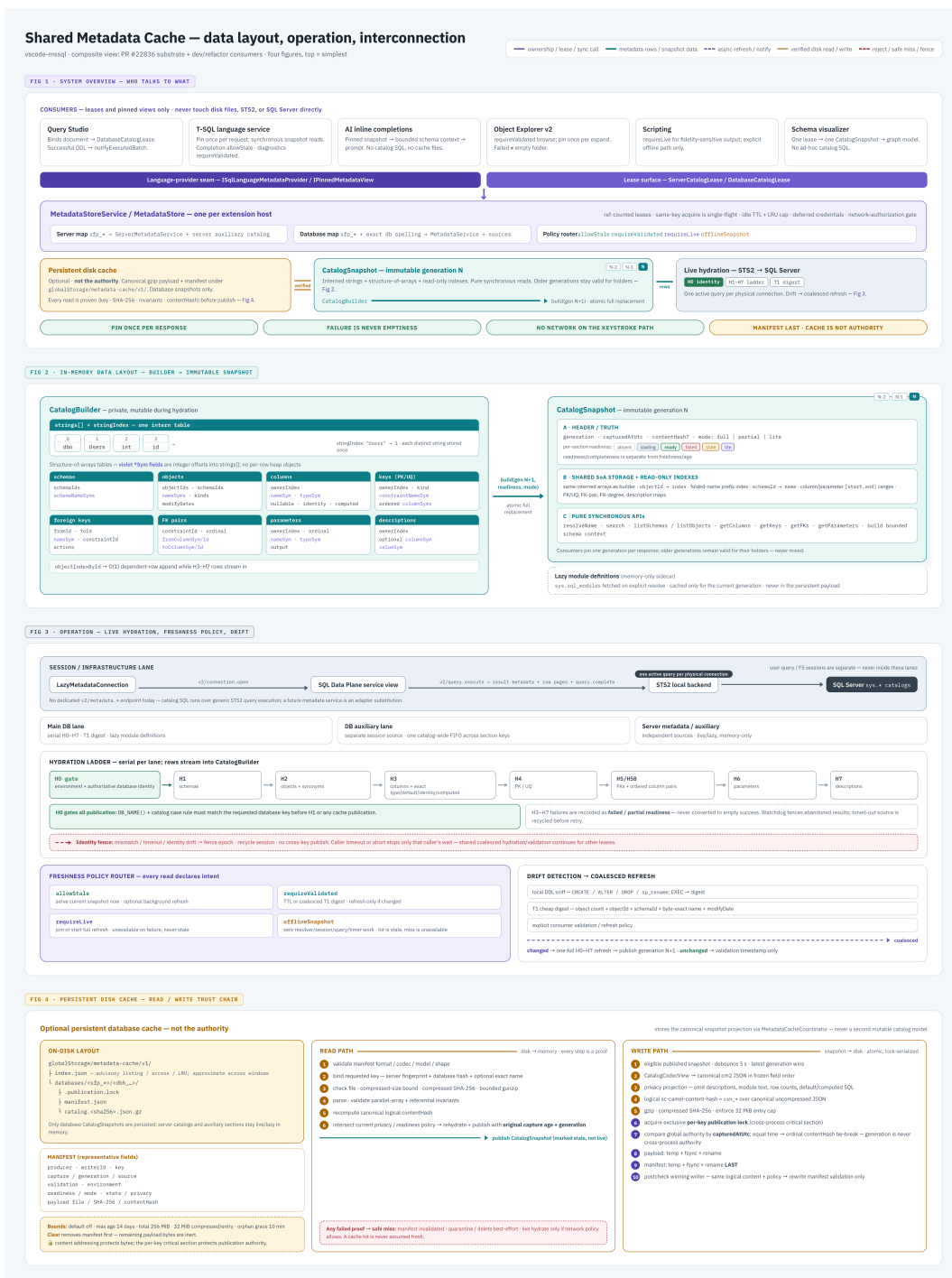


Figure 11: Alternate tall composite rendering. The four stacked panels progress from system interactions to in-memory structure, live acquisition and freshness, and finally the persistent-cache read/write protocol. This form is convenient for scrolling and for reading the architecture as one continuous pipeline.

Figure 12: Contact sheet for the focused diagram set. Clockwise from upper left: shared architecture and consumer flow, immutable catalog layout, persistent-cache trust and publication, and acquisition/freshness/drift. The following pages reproduce each panel from its vector PDF source.

Shared Metadata Substrate: Architecture and Consumer Flow

Composite view: PR #22836 substrate below; downstream dev/refactor consumers above (not cut over by this PR).

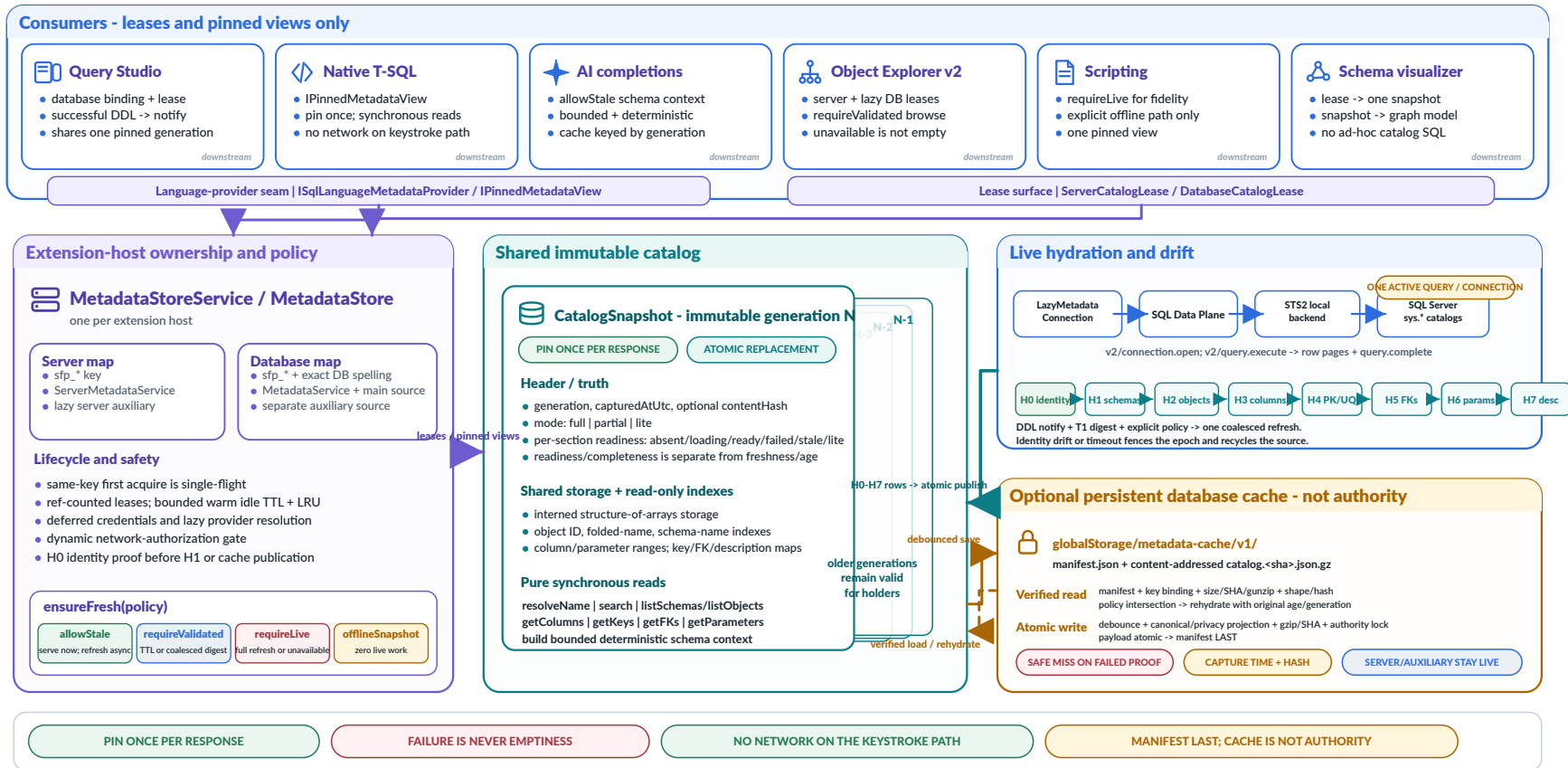


Figure 13: Shared metadata architecture and consumer flow. The PR #22836 substrate owns key-correct leases, immutable catalog snapshots, live hydration, and optional persistence. Downstream language, AI, Query Studio, Object Explorer, scripting, and visualization consumers use leases or pinned views and do not bypass the shared store.

Immutable Catalog Data Layout

SQL catalog row pages are accumulated in a private structure-of-arrays builder, then published as one immutable generation.

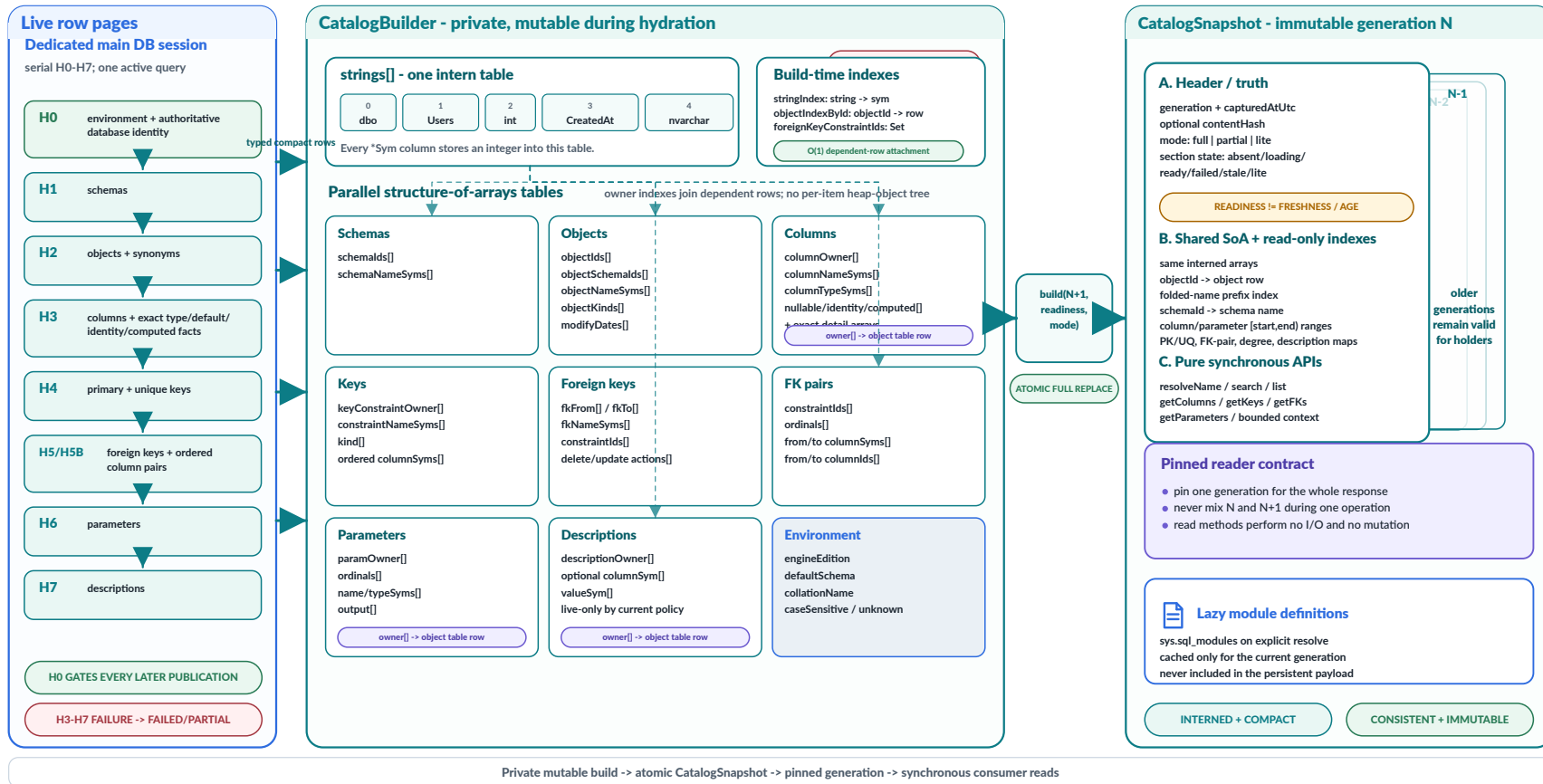


Figure 14: Compact immutable catalog layout. H0–H7 catalog rows accumulate in a private structure-of-arrays builder with one interned string table and owner indexes. `build()` atomically publishes a `CatalogSnapshot`; readers pin one generation and use pure synchronous APIs.

Metadata Acquisition, Freshness, and Drift

Cache-first acquisition and live refresh share one rule: return the best snapshot with an explicit truth label, never an empty lie.

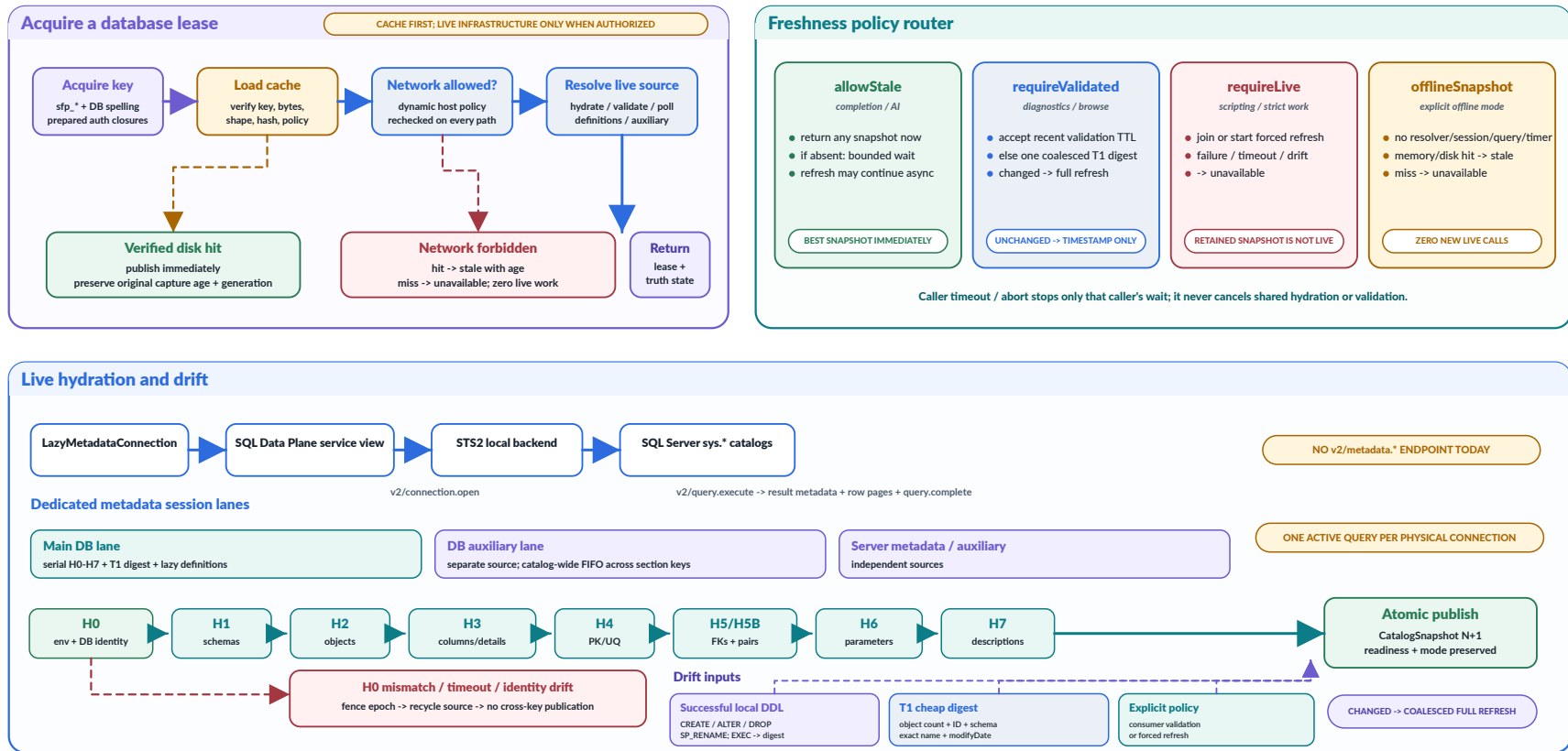


Figure 15: Acquisition, freshness, and drift. Acquisition first attempts a verified local snapshot and resolves live infrastructure only when network policy permits. Consumers express freshness intent through `allowStale`, `requireValidated`, `requireLive`, or `offlineSnapshot`; drift signals coalesce into a full H0–H7 refresh.

Persistent Catalog Cache: Trust and Publication Protocol

The cache is an optional projection of a published database snapshot. It is verified local input, never catalog authority.

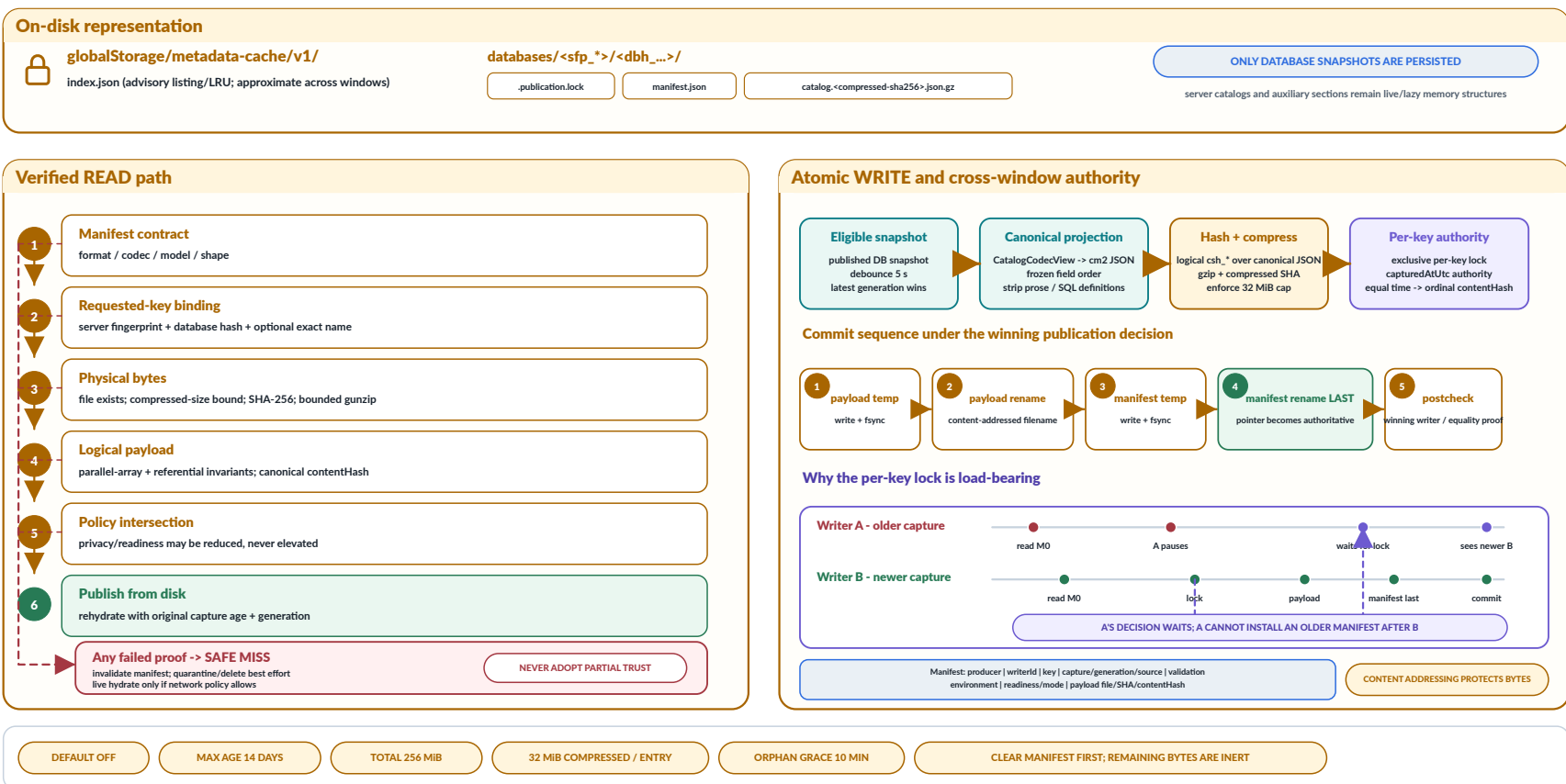


Figure 16: Persistent-cache trust and publication protocol. A disk entry is adopted only after manifest, key, byte, logical-shape/hash, and policy checks. Writes use canonical privacy-projected payloads, content addressing, a per-key authority lock, atomic payload replacement, and manifest-last publication.

10 Language-service fit

10.1 The contract supplied by this PR

The future native language engine needs stable environment and catalog views, not direct knowledge of cache files, SQL sessions, or mutable hydration. The metadata substrate supplies the raw ingredients:

- environment: current/default database/schema, catalog case behavior, engine edition, collation;
- name resolution: exact/default-schema/ambiguous/not-found/section-unavailable;
- schemas, objects, columns, keys, FKs, parameters, descriptions, search and system/auxiliary projections;
- a generation that invalidates per-request derived caches; and
- explicit readiness/freshness so diagnostics can suppress instead of hallucinating.

The language-service design in the notes adds a thin provider interface and an `IPinnedMetadataView`. That provider should adapt `CatalogSnapshot` without exposing implementation classes. Every feature request pins once; all synchronous completion/binding operations read that generation only. Lazy module definitions remain the exceptional asynchronous resolve path.

10.2 No network I/O on the keystroke path

Completion, hover, signature help, and diagnostics should never await H0–H7 while the editor is responding. The intended pattern is:

1. acquire/retain a database lease when a document/connection binding becomes active;
2. call an appropriate freshness policy outside or at the bounded edge of interactive work;
3. pin one snapshot for the feature request;
4. answer synchronously from it, including local overlay symbols; and
5. request hydration/refresh asynchronously when data is absent or stale.

Completion can use `allowStale` and surface available candidates immediately. Diagnostics use `requireValidated` with a short budget and suppress schema claims if columns/objects are unavailable. Scripting uses `requireLive` because emitting destructive or fidelity-sensitive DDL from stale metadata is unacceptable.

Table 7: How planned language features consume metadata honesty.

Feature	Suggested policy	Required sections	Degradation
Completion	<code>allowStale</code>	objects; columns when member access	Offer known candidates; omit unavailable detail; refresh in background.
Hover/signature	allow stale or validated	objects, columns, parameters	Show identity/type that is known; do not fabricate docs or parameter lists.
Diagnostics	<code>requireValidated</code>	objects + columns	Suppress schema diagnostics on timeout/-partial metadata; syntax diagnostics remain independent.
Definition	validated + lazy resolve	objects; module definition on resolve	Navigate to stable synthetic anchor; fetch module text only on explicit resolution.
Scripting	<code>requireLive</code>	all fidelity-critical sections	Refuse or offer explicit offline/stale path with fidelity note.
AI context	<code>allowStale</code>	bounded projection	Use deterministic, size-bounded context; include readiness, never prompt with silent gaps.

Scope boundary

PR #22836 does not instantiate this language provider or switch the existing SqlToolsService language client. Its only language-client change is to gate STS2 launch arguments through the private-preview predicate. Language correctness therefore belongs to the following consumer PR, while catalog/provider correctness belongs here.

11 STS2 integration and endpoint evolution

11.1 What STS2 actually exposes today

At `upstream/dev/query` commit `fe8300939`, STS2 shares stdio with the legacy service. The multiplexer routes `v2/*` methods to STS2 and leaves other traffic on the legacy route. The generated v2 contract contains `initialize`, `diagnostics`, and `fatal-notification` methods; `connection open/cancel/close` methods; and `query execute`, `result-set`, `rows`, `message`, `complete`, `acknowledgement`, `cancel`, and `dispose` methods.

There is no `v2/metadata.*` method in that contract. The `vscode-mssql` capability registry correspondingly reports:

- `metadata.catalogSql`: supported;
- `metadata.endpoints`: unsupported with reason `notImplemented`.

The metadata engine therefore opens dedicated STS2 connections and sends fixed catalog SQL through `v2/query.execute.runMetadataQuery()` collects compact row pages, treats server error messages as failure, waits for the matching query handle completion, and only then starts the next H-step. The STS2 adapter owns query IDs, page acknowledgements, terminality, and disposal; the metadata layer sees the data-plane session abstraction.

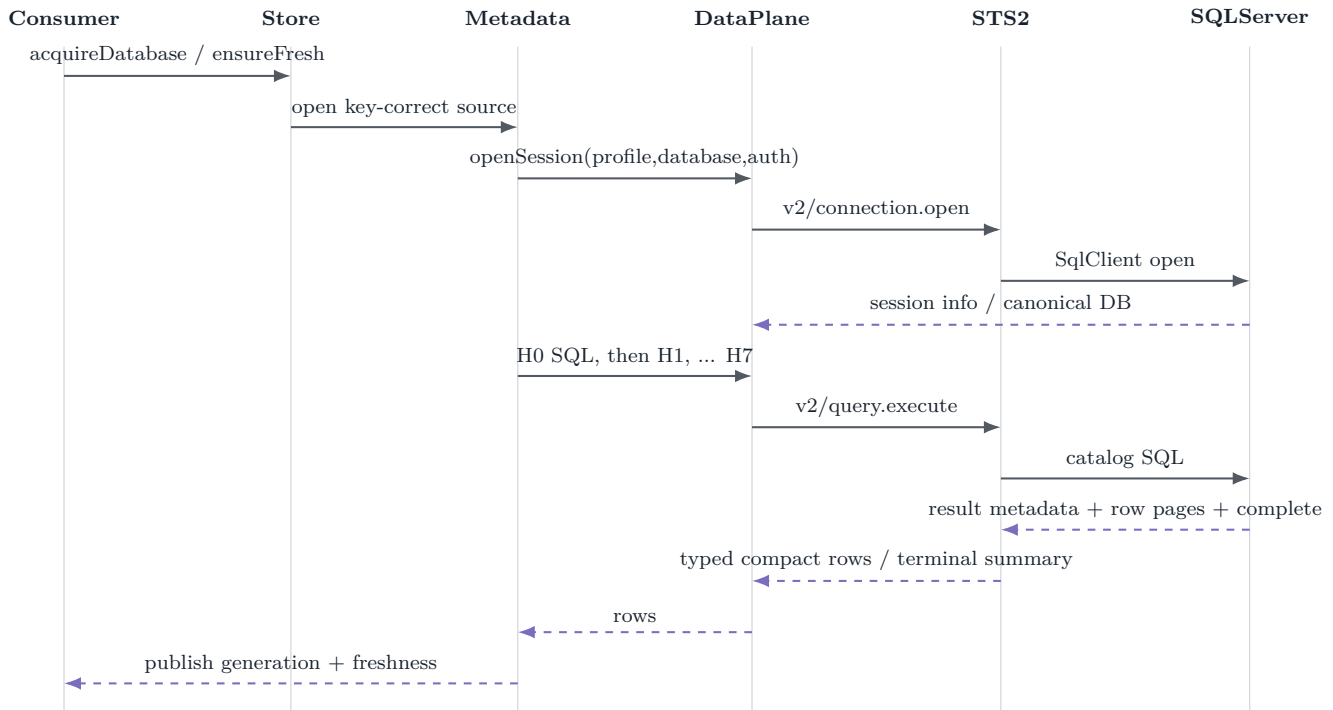


Figure 17: Current end-to-end path. The metadata service is a catalog-aware client over generic STS2 query execution, not a server-side metadata RPC.

11.2 Why dedicated sessions are the right bridge

STS2 permits one active query per connection and enforces page-window backpressure and exactly one terminal completion. The database engine correctly serializes its H-ladder, digest, and lazy module work on one session. User query sessions remain independent, so main-lane catalog work does not race with F5 execution.

The auxiliary boundary is only partially complete: server/database auxiliary catalogs have separate sources from the main lane, but different auxiliary section keys can overlap through the same source's cached session. The executable STS2 scenario `second-query-while-active-busy.yaml` expects the second execute to fail with `Sts2.Busy`. A per-catalog FIFO lane is therefore part of the adapter contract, not optional optimization.

11.3 One owner for serverless wake and retry

The SQL Data Plane has two public entry shapes: top-level `SqlDataPlaneService.openSession()` and a service view returned by `service()` or `serviceForProfile()`. The current top-level method obtains the public view and wraps `view.openSession()` in `openWithServerlessWake()`; the view's `openSession()` also wraps the raw provider call in the same helper. The resulting call graph is outer wake → inner wake → backend open.

Nested wake policy can multiply ARM status checks, delay windows, and open attempts, and makes attribution ambiguous. Choose one owner. The simplest option is for the public view to own requirements, provider availability, session registration, and wake, while the top-level method delegates directly after its pure capability/fallback decision. A spy-based unit test should assert one wake invocation and the exact backend open-attempt count for paused, resuming, and ordinary profiles.

Related open SQL Data Plane thread

A current Copilot thread also notes that a failed provider `canOpen()` result is routed through the capability-unsupported error helper, even when the failure represents availability/handshake state. Keep that classification fix separate from the duplicate-wake repair: capability mismatch, backend unavailability, and serverless resume are different error/retry domains.

11.4 How dedicated metadata endpoints can fit later

A future STS2 contract may add endpoints that return server/database sections directly, potentially with server-side paging, digests, or change tokens. The clean migration point is the `MetadataSessionSource`/data-source adapter:

1. keep `CatalogSnapshot`, store keys, lease lifecycle, freshness policies, persistence, and consumer provider unchanged;
2. add a source that checks `metadata.endpoints` and calls the new methods;
3. retain catalog-SQL fallback only where its declared fidelity is sufficient; and
4. prove endpoint-vs-SQL provider equivalence with the same snapshot fixtures and invariant suite.

The endpoint should not smuggle consumer policy into STS2. The server can own efficient catalog acquisition and stable wire shapes; the extension should continue to own cross-consumer leases, local persistence policy, VS Code focus/offline behavior, and language/UX degradation.

12 Logging, diagnostics, and privacy

12.1 Extension-side event vocabulary

The feature emits through the unified `diag` core, where every field is classified before reaching live-tail, session storage, perfest, or self-test sinks. Principal event families are:

Table 8: Metadata observability surface at the extension layer.

Family	Representative events	Safe diagnostic meaning
Engine	<code>metadata.hydrate</code> , <code>metadata.ensureFresh</code> , and validation/drift events	Duration, generation, policy mode/reason, freshness/source, wait, error class; database fields are classified paths.
Store	acquire server/database, dispose lease, server/aux hydration, access drift, key-correctness tripwires	Short fingerprint, key kind, cache hit/miss, refcount, generation/readiness/counts; no catalog item names.
Cache	<code>load</code> , <code>save</code> , <code>hit</code> , <code>miss</code> , <code>corrupt</code> , <code>evict</code> , <code>clear</code> , and <code>raceLost</code>	Fingerprint/hash prefixes, byte/duration counts, fixed age buckets, safe enums, policy-intersection boolean.
Performance	<code>mssql.metadata.cache.warmAcquire.begin</code> / <code>mssql.metadata.cache.warmAcquire.end</code>	Same-process monotonic marker pair; object count and wait time; feature bucket <code>metadata</code> .

The generated observability registry is the source of truth, mirrored into the embedded perfest package. Conformance tests check that source-emitted names are registered and marker pairs agree.

`Perf.featureFor()` maps `metadata.*`, `metadataStore.*`, `metadataCache.*`, and `mssql.metadata.*` to the metadata feature.

12.2 Allowlist and forbidden fields

Allowed event attributes include short non-reversible fingerprint/hash prefixes, generations, readiness summaries, fixed stale-age buckets, byte/duration/wait counts, safe enum results/reasons/tiers, booleans, and error classes. Forbidden fields include raw object/database names unless classified and redacted at the common choke point, SQL text, result rows, endpoints, connection strings, secrets/tokens, prompt text, module definitions, description values, and content-hash prefixes pending policy review.

The key-correctness tripwires intentionally classify expected/actual database values as `source.path`; they do not bypass the central redactor. These events prove that a mismatch was detected, not that publication was prevented. Cache status/clear UI may display local database names to the user, but those names do not become telemetry.

12.3 Extension diagnostics versus STS2 envelopes

These are complementary layers:

- **Extension diagnostics** explain consumer/store/cache decisions: why a snapshot was served, rejected, refreshed, or evicted.
- **STS2 envelopes** explain transport/runtime causality: RPC input, reducer output, effects, query pages, terminal events, configuration changes, and runtime health.

STS2 journals each sanitized envelope before dispatch, then publishes it in sequence order to best-effort metrics/live-tail/custom sinks. Product capture replaces SQL text and row cells with digest wrappers; secret material lives only in an in-memory side table behind opaque references. The envelope carries run ID, gapless sequence, UTC timestamp, kind/type, session/correlation, causal sequence, config version, payload digest, optional payload, and redaction metadata.

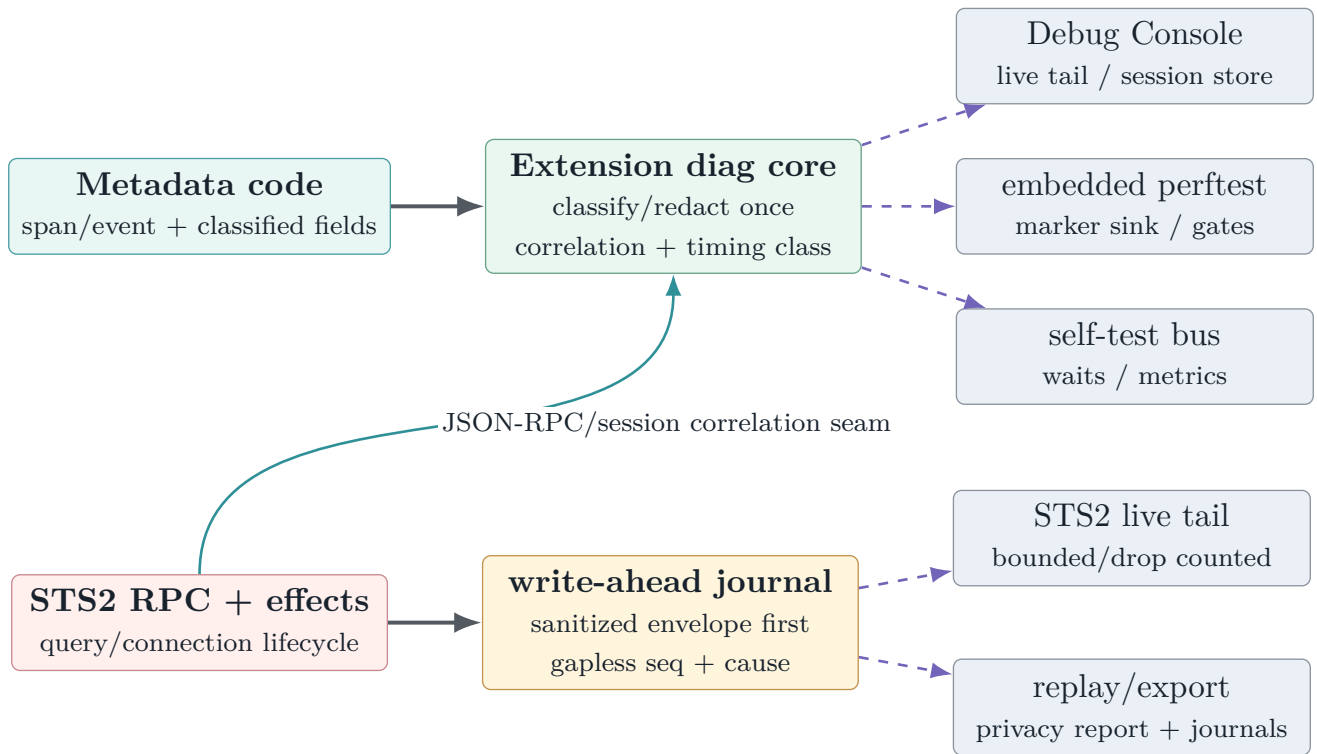


Figure 18: Two observability planes. Extension events explain metadata policy; STS2 envelopes explain transport causality. Both are bounded and privacy-classified, and can be stitched without copying SQL or rows into product telemetry.

13 Test design and coverage

13.1 Strength of the current test strategy

The focused suite is unusually good at semantic failure modes: immutable publication, section readiness, deterministic ordering, exact catalog detail, same-key acquisition, lifecycle disposal, codec bounds, malformed payloads, torn writes, privacy canaries, poll governance, and large-catalog scaling. Prior review comments also drove concrete fixes for quadratic projection, delimiter collisions, UDT rendering, hidden columns, role-owned schemas, untrusted FKs, decompression bounds, content-addressed payloads, and shutdown flush.

The original review found that test volume was high but several unsafe branches were not selected. The repaired head adds those adversarial boundaries directly, so the closure claim rests on exact interleavings and call-count assertions rather than nearby happy paths.

13.2 Current focused suite inventory

The correctness-closeout run at `e5952174a` reported 158 passing tests across the metadata/cache/-catalog files below, including the new auxiliary-catalog suite; adding touched profile-identity, private-preview/manifest, and SQL Data Plane files yielded 206. After merging current `main` and adding activation-snapshot coverage, the combined `Metadata|AuxiliaryCatalog|SQL Data Plane|PreviewFeaturesService|Private preview manifest|Extension API Tests` selection passed 260/260 at `37bf23fce`. The final public-view alternatives fix then passed its owning backend-registry suite

17/17 at 69ca292f8. These are executed-inventory counts, not a claim that every assertion in pre-existing touched files was introduced by this PR.

Table 9: Metadata closure inventory introduced by e5952174a and re-executed within the merged-head selection.

Suite	CI count	Principal invariants
metadataCatalog	36	H-ladder parsing, readiness/partial failures, resolution/search, exact-only unknown-case behavior, DDL keyword detection, descriptions, lazy definitions, burst coalescing, hidden columns, safe IDs.
metadataCatalogDetail	6	Exact column/FK facts, UDT spelling/schema, identity text, defaults/computed detail, FK actions, and column identity.
metadataDeterminism	6	Ordinal ordering, byte-stable schema context, catalog-collation case behavior, and digest-v2 rename detection.
metadataFreshness	10	Four policy modes, TTL, timeout-as-race, abort-listener balance, server watchdog/recycle recovery, section gates, and stale/live/unavailable honesty.
metadataPoll	12	Backoff, jitter bounds, focus suspension/resume, serverless cap, limiter behavior, and failure retry.
Governance		
metadataStore	13	A/B key isolation, same-key single flight, routed sessions, leases, refcounts, idle TTL/LRU, cleanup, and server catalogs.
metadataStoreCache	6	Disk-before-live publication, manifest generation/digest baseline, background refresh, offline seeded hit, recovery, and A/B isolation with cache.
metadataStoreScale	6	10k-object hydration/counts, ordering/search, 1k-column table, bounded schema context, and warm reacquire without rehydration.
metadataStoreService	3	Default-off composition, enabled coordinator, and data-plane-disabled fail-fast without backend acquisition.
metadataCacheCodec	20	Live/disk round-trip equivalence, byte stability, content-hash perturbation, privacy stripping, strict shape/range/ownership validation, and model/version misses.
metadataCacheStore	39	Path containment, requested-key and logical-hash integrity, oversize bounds, quarantine, rename retry, torn writes, older-last and manifest-only/full writer races, eligibility/skips, policy intersection, privacy canaries, index/eviction/clear/settings.
auxiliaryCatalog	1	Two distinct sections started together complete FIFO through one one-active-query session.
Metadata subtotal	158	Repaired-head local hosted counts; zero failures.

This inventory now includes the stronger boundary proofs. The regression matrix below records how each prior finding was closed and remains the checklist to retain through future refactors.

Table 10: Regression matrix closing the review findings at final head 69ca292f8.

ID	Invariant	Prior gap	Repaired-head proof
F1	Offline setting means zero live work	Seeded hit with polling disabled	Hit, miss, refresh, strict policies, auxiliary, polling, and online→offline assert resolver/session/query counts remain zero.
F2	Wrong database never publishes; drift/timeout reopens	Correct routing and later digest drift	H0 returns WrongDb ; no snapshot publishes, violation latches, and retry opens a fresh session through forwarded recycle() .
F3	One active query per auxiliary session	Same-section single flight	Two distinct delayed sections start together; both become ready through one fake one-active-query session.
F4	Cache entry belongs to requested key	Path derivation/containment	A valid A manifest/payload copied into B's directory returns keyMismatch and is invalidated.
F5	Newer authority cannot be over-written	Post-save winner observation	Per-key lock tests force older-last and delayed manifest-only/full interleavings; capture time beats local generation and new data remains authoritative.
F6	One serverless wake policy per open	Registry/view tripwires	Top-level open reaches the provider exactly once; the existing eight-case wake suite pins eligible retry bounds; provider refusal is retryable unavailable.
F7	Wait abort listener is removed	Server service only	Database service proves add/remove balance after success and timeout while reusing the same signal.
F8	Unknown case rule never folds optimistically	H0 success and collation divergence	Unknown semantics resolve exact spellings independently and reject a folded-only spelling.
F9	Server refresh recovers from hung completion	Caller timeout race	Swallowed server completion times out, publishes failed/not-empty truth, recycles, and succeeds on the next refresh; epoch fencing blocks late mutation.
F10	Manifest logical hash matches decoded payload	SHA and canonical hash separately	Valid compressed SHA and payload shape with a false contentHash returns corrupt and quarantines.

13.3 Persistence tests remain protocol tests

Cache tests should continue to model filesystem operations as a protocol, not as ordinary serialization coverage. The completed suite already covers many crash points and corruption classes. The next additions must distinguish three layers:

- **byte safety**: size, SHA, gzip bound, JSON and array invariants;
- **identity safety**: requested key and logical content hash;

- **authority safety**: one atomic, globally comparable publication decision per key.

A result such as `raceLost` is only meaningful if the final manifest is known to represent the winning authority. Post-save writer-ID comparison alone does not prove that when an older writer installs last.

13.4 Embedded perftest coverage

The embedded scenario `metadatabacache-warm-acquire` is the correct performance boundary: cold fill/save, a fresh store in offline mode, disk verification and rehydration/publication, source-honesty assertion, and begin/end markers around only the warm acquire. It is exploratory rather than an official SLA gate. Before graduation it should run across representative catalog sizes, Windows/macOS/Linux, local and remote extension hosts, and repeated cold/warm profiles with pre-registered p50/p95 budgets.

The repaired offline unit matrix asserts resolver calls and session opens directly rather than relying only on a `disk` source label. The embedded performance scenario should retain the same counters if promoted from exploratory measurement to an SLA gate.

Local package verification in this work session built the embedded perftest workspace and ran 347 tests passing with 16 explicit skips. Fifteen skips are central-store integrations and one is vendor-sync; the Query Studio scenario tests (158) and VS Code spawn tests (2) passed. This verifies registry/contracts/harness behavior, not a live SQL warm-cache benchmark run against a provisioned `PerfHarness` database.

13.5 Coverage evidence and its limit

Codecov at the pinned head reports 91.09384% patch coverage with 894 changed lines uncovered. Project coverage rises from 77.17% at the PR base to 77.96% at the head. Core metadata files are strong—catalog model 97.22%, codec 95.81%, coordinator 96.20%, store 94.47%, and engine 93.44%—while host wiring and broad auxiliary surfaces are weaker: main controller 9.38%, database auxiliary sections 73.75%, auxiliary engine 77.44%, and store composition service 86.53%.

Coverage supports the focused suites but does not settle the review findings. Race ordering, dynamic network authorization, key binding, and retry ownership are semantic properties; they require the deterministic adversarial tests in [table 10](#), not merely more executed lines.

14 Build and validation record

14.1 Current GitHub state observed independently

At prior head `9955ff9e6`, analysis/CodeQL, Azure PR validation, packaging, normalization, CLA, and all six platform VSIX-launch jobs passed. Its GitHub `build-and-test` check was red, but its metadata-relevant facts were more specific:

- extension install/build, lint, package, and MSSQL unit-test steps pass;
- the GitHub unit runner reports 3,975 passing, 0 failing, and 12 skipped;
- every metadata file in [table 9](#) reports zero failures;
- the aggregate job fails because SQL Projects unit tests return exit code 1 and one of 17 VSIX smoke tests—“MSSQL add connection webview is loaded correctly”—fails.

This was not evidence of a metadata failure. The later correctness head became conflicting as `main` advanced. Merge commit `37bf23fce` incorporates current `main` and resolves that conflict; its build/test

workflow passed. Five of six platform launch jobs also passed. The sole win-x64 failure occurred in test-dependency installation: `npm ci` reported Windows `EPERM` cleanup failures under `monaco-editor`, then `git-dependency` preparation exited nonzero. Artifact download and the extension-launch step were skipped, so that red workflow contains no product-runtime failure. Final head `69ca292f8` adds only the reviewed public-view alternatives repair. Its analysis, aggregate CodeQL, normalization, CLA, MSSQL VSIX packaging, aggregate build/test, and linux-x64, linux-arm64, macOS-x64, macOS-arm64, and win-x64 launches passed. Win-arm64 launch and the separate Azure PR validation context remained in progress at report freeze; live PR checks remain the authority for the eventual merge button.

14.2 Independent local validation

The local workspace was updated and built at the pinned commits before this report. Results are kept separate from PR-author comments and GitHub CI:

Table 11: Independent local validation snapshot, updated 29 August 2026.

Lane	Result	Interpretation / explicit exclusions
vscode-mssql build	Pass	Full monorepo build repeated on 69ca292f8: shared toolkit, MSSQL extension/typecheck/bundles/webviews/notebook renderer, merged SQL Projects runtime, legacy SQL Database Projects package, and Data Workspace.
vscode-mssql lint	Pass	Root lint exited zero for extension-toolkit, MSSQL, SQL Database Projects, and Data Workspace.
Focused tests	260 pass; 0 fail at 37bf23fce; 17 pass; 0 fail at 69ca292f8	Broad metadata/cache/catalog, auxiliary, profile/preview/manifest, SQL Data Plane, and activation-context selection followed by the owning backend-registry suite for the final alternatives fix.
MSSQL broad hosted tests	4,025 pass; 0 fail; 10 skip at merge parent 37bf23fce	Complete inverse selection excludes exactly QueryCompletionSoundService and Utility Tests - Path handling . The full run otherwise reports eight failures, all reproduced alone as untouched Windows path-separator expectations (seven sound, one tilde path). The final change is restricted to returning already-computed alternative identifiers and its focused suite, full build, and full lint were rerun.
SQL Projects hosted tests	205 pass; 0 fail; 6 skip	Current-main isolated SQL Database Projects unit lane, run in a separate VS Code host as required by the merged test configuration.
Localization	Pass	Root extraction processed all four projects and extracted 3,064 MSSQL strings; commit hooks repeated extraction and lint-staged successfully.
Direct packaging	Pass	MSSQL produced 2,453 files / 208.53 MiB with the worktree's existing 617.8 MiB expanded STS payload. SQL Projects produced 428.41 KiB, Data Workspace 17.10 KiB, and the keymap 7.14 KiB. The root wrapper cannot forward MSSQL-only -online -skip-service-install through SQL Projects because vsce rejects -online ; direct workspace packages prove the artifacts themselves.
Embedded perftest	347 pass; 16 skip	All package tests/builds pass; skips are described above; the live provisioned warm-cache scenario was not executed.
sqltoolsservice Release solution	Pass; serial MSBuild with shared compilation disabled	0 errors; two ScriptDom locale warnings.
STS2 unit + E2E	496 pass / 10 SQL-server-gated skip; 6/6 E2E	Deterministic/runtime contract and process interop green; engine-gated skips explicitly reported.
STS auth / language	9/9; 76/76	Adjacent authentication and current language-service lanes green.
ServiceLayer unit	1,944 pass; 1 fail	One platform/provider DNS failure, not metadata logic. No on-prem integration lane was provisioned.

Evidence boundary

The local runs establish that the repaired tree builds/packages and close F1–F10 with deterministic unit regressions. They do not replace live SQL compatibility testing, the provisioned warm-cache

scenario, or on-prem/remote-host performance matrices; those remain graduation gates rather than merge blockers for this default-off substrate.

14.3 Author- and review-reported evidence

The PR body reports additional focused host-composition, data-plane capability, observability, localization, packaging, and full-suite results. Prior review threads report targeted performance probes and repeated focused suites after earlier fixes. These are useful confidence signals and should remain in the PR record, but they are **Reported evidence** rather than rerun as part of this review pass.

All 52 review threads are resolved at the pinned head with source-attributed dispositions. Provider startup refusal is retryable `unavailable`; the QuickPick separates localized capture text from fingerprint detail; activation-snapshot keys prevent pre-reload command exposure; and bound service views retain registry alternatives on a requirement failure. The CR-only line-comment suggestion was declined at the raised closeout threshold: VS Code documents support LF/CRLF, both are handled, and a synthetic CR-only miss would merely defer the best-effort DDL refresh to digest polling.

15 Performance mechanisms and evidence quality

The implementation removes the most important algorithmic traps:

- constant-time object-ID lookup during build;
- value-by-value row-page append rather than spreading unbounded arguments;
- snapshot indexes for name/prefix lookup;
- linear schema-context candidate rendering per fidelity tier;
- same-key acquisition, validation, and forced-refresh coalescing;
- content-equal cache saves reduced to a manifest update;
- delayed/throttled maintenance outside activation;
- focus-gated, backoff/jittered, per-server-limited polling.

Prior review measurements show that the fixed scale paths are orders of magnitude better than the earlier quadratic implementation. Those measurements are review-machine observations, not product SLAs. The repaired single wake owner and auxiliary FIFO remove the two policy-multiplication risks identified in the prior review; lock contention and cache-load latency should still be measured across local/remote filesystems before enabling persistence by default.

A graduation performance plan should separate:

1. live H0–H7 hydration CPU, SQL time, memory peak, and payload size;
2. warm cache read, hash/decompress/validate, and publication time;
3. interactive snapshot query/projection latency with no network;
4. background poll/validation cost under focus, resume storms, and serverless pause; and
5. multi-window contention/lock hold time after the authority fix.

16 Mechanism-to-evidence traceability

Table 12: Load-bearing mechanisms and repaired-head closure evidence.

Mechanism	Current evidence	Disposition
Same-key single flight	Eight-way concurrency tests and explicit disposal coverage	Retain.
Immutable deterministic snapshot	Catalog/detail/determinism/round-trip/scale plus unknown-case exact-only regression	Closed; retain.
Database identity	H0 mismatch-before-publication, violation latch, recycle/reopen and disposal-race tests	Closed; live SQL collation/database corpus remains a graduation gate.
Offline behavior	Dynamic resolver/engine/server/auxiliary gate with hit/miss/transition call-count matrix	Closed.
STS2 one-query discipline	Main lane, catalog-wide auxiliary FIFO, database/server watchdog and recycle tests	Closed.
Cache byte integrity	Content addressing, SHA/gzip/shape, requested-key copy test, canonical logical-hash test	Closed.
Cache cross-window authority	Per-key lock, capture-time ordering, older-last and manifest-only/full interleaves	Closed for catalog authority; advisory index LRU remains documented approximate.
Privacy at rest	Codec stripping, manifest policy, disk canaries rerun after cache changes	Closed; retain.
Data-plane capability/auth	Pure tripwires, retryable availability classification, one top-level provider open, wake suite	Closed.
Consumer seam	Lease/snapshot/freshness APIs and refactor designs	Future provider equivalence, pinned-generation, no-keystroke-I/O, and honesty corpus.

17 Known boundaries and next validation gates

17.1 Model enrichments deliberately deferred

The substrate reports rather than conceals incomplete domains. Synonym target traversal, table-type shapes, sequences, parameter/schema descriptions, row counts, and section/object-level T2/T3 digests need new model/API shapes. They should land with consumers that can exercise them end to end, not as unreachable arrays that misleadingly say **ready**.

17.2 Integration work for the native language service

The following gates should precede a language cutover:

1. retain the now-closed F1–F3 regressions so offline, identity, and STS2 lane contracts remain trustworthy;
2. implement the provider adapter and prove fixture/null/catalog provider equivalence;
3. pin one generation per feature request and test a mid-request generation change;

4. prove completion/hover/diagnostics make no network call on the keystroke path;
5. run the honesty corpus: temp tables, CTEs, dynamic SQL, synonyms, cross-database names, case-sensitive catalogs, partial/offline metadata, and USE changes;
6. define suppression/status UX for unavailable sections and stale/offline snapshots;
7. benchmark real completion p50/p95 and document CPU/memory budgets separately from cache load.

17.3 Integration work for dedicated STS2 endpoints

Before flipping `metadata.endpoints` to supported:

1. add generated wire-contract methods and stable error shapes in STS2;
2. define section/readiness semantics and paging/backpressure for large catalogs;
3. preserve database-key correctness and one-terminal cleanup under cancel/dispose;
4. decide whether STS2 returns canonical arrays, row-like sections, or a provider-neutral DTO—without importing the extension cache format as wire format;
5. produce a live-SQL truth corpus and endpoint-vs-catalog-SQL snapshot-equivalence tests;
6. extend STS2 envelope/privacy canaries so metadata names and definitions follow explicit capture policy.

17.4 Completed closeout plan

Table 13: Completed repair and merge-closeout order for PR #22836 at 69ca292f8.

Order	Repair	Closure status
1	F2 fail-closed identity + recycle forwarding	Implemented; mismatch cannot publish and later retry reopens.
2	F1 dynamic offline network gate	Implemented across store, engines, timers, definitions, and auxiliary paths.
3	F3 auxiliary FIFO + F9 server watchdog	Implemented and regression-tested.
4	F4 key binding + F10 logical-hash verification	Implemented and corruption/copy-tested.
5	F5 per-key cross-window authority	Implemented with exclusive-create lock and adversarial interleavings.
6	F6 single wake owner + availability classification	Implemented; registry/conformance/wake suites pass.
7	F7 abort listener + F8 unknown case behavior + index documentation	Implemented; advisory cross-window LRU explicitly accepted as approximate.
8	Full focused suites, broad hosted suite, build/lint/package, Actions rerun	Local gates complete; fresh GitHub Actions triggered. Live SQL/performance matrices remain pre-graduation work.

18 Final assessment

Key takeaway

The architectural direction is good: one shared lease/store, immutable compact snapshots, explicit per-section truth, policy-routed freshness, default-off persistence, content-addressed payloads, and a future transport/provider seam are the right foundations for native language and AI-assisted SQL experiences.

At 69ca292f8, the implementation enforces the design narrative's load-bearing claims: zero-network offline operation, key-correct publication, one-active-query auxiliary behavior, newer-writer cache authority, decoded logical integrity, bounded hydration recovery, one serverless wake owner, activation-consistent preview UI, and capability-alternative parity across public data-plane views. The feature remains private-preview/default-off and no ordinary language, AI, OE, or Query Studio surface is cut over by this PR.

Decision statement

Recommended disposition: code/design review findings are closed; proceed to CI and human merge review. Do not enable the substrate by default or cut over consumers based only on these unit/build results. Live SQL compatibility, remote-host cache behavior, and real workload performance remain graduation gates for later preview stages.

Evidence and source map

Reproducibility note

This report is pinned because the PR and related branches are active. Review threads, Actions conclusions, and code locations are observations at the stated snapshot, not timeless properties. Re-fetch the head, rerun the specified regression and whole-PR lanes, and update the title-page pins and disposition before using the PDF after any new commit.

Evidence class	Primary sources used
PR state	PR #22836 , current body/discussions/check metadata, merge-ready head 69ca292f8, current-main merge 37bf23fce, main parent 59c8ea50c, extraction base 69f54be2a.
Core implementation	extensions/mssql/src/services/metadata/catalogModel.ts; metadataService.ts; metadataStore.ts; metadataStoreService.ts; serverMetadataService.ts; auxiliaryCatalog*.ts; profile/auth/fingerprint modules.
Cache implementation	extensions/mssql/src/services/metadata/cache/metadataCacheCodec.ts; metadataCacheManifest.ts; metadataCacheStore.ts; metadataCacheCoordinator.ts; metadataFreshness.ts; metadataCacheSettings.ts.
Host/data plane	mainController.ts; previewService.ts; sqlDataPlaneService.ts; languageservice/serviceclient.ts; package settings/localization; perfTelemetry.ts.
Tests	Metadata/cache/catalog/freshness/poll/store/scale suites, profile and data-plane tests, observability conformance, large-catalog fixture, and in-memory filesystem harness; CI per-file counts plus the independent local validation in table 11 .

Embedded perftest	<code>tools/perftest/packages/perftest-cli/src/scenarios/registry.ts</code> and the metadata warm-acquire command/marker wiring.
Implementation review	Companion workspace review <code>vscode_mssql_pr_22836_code_review.md</code> ; every finding was rechecked against local and pushed head <code>69ca292f8</code> before publication.
Metadata/refactor designs	<code>vscode-ext-refactor/metadata-docs/*</code> , including substrate, cache/drift, and review addendum documents; current code controls when design notes and implementation differ.
Language/observability designs	Native T-SQL language-service and observability notes under <code>vscode-ext-refactor</code> ; treated as downstream contracts, not PR content.
STS2 authority	<code>microsoft/sqltoolsservice dev/query</code> at <code>fe8300939</code> : generated contract, <code>docs/sts2/INVARIANTS.md</code> , runtime/client documentation, and <code>test/sts2/scenarios/second-query-while-active-busy.yaml</code> .
